

Universidade Federal de Viçosa
Campus Rio Paranaíba

Arthur Roberto de Paula Neto - 8079

Análise Comparativa de Algoritmos de Ordenação

Rio Paranaíba - MG

2025

Universidade Federal de Viçosa
Campus **Rio Paranaíba**

Arthur Roberto de Paula Neto - 8079

Análise Comparativa de Algoritmos de Ordenação

Trabalho apresentado para obtenção de créditos na disciplina SIN213 - Projeto de Algoritmo da Universidade Federal de Viçosa - Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba - MG

2025

1 Resumo

A análise de algoritmos de ordenação é fundamental para o entendimento de diferentes abordagens e estratégias utilizadas para reorganizar dados de forma eficiente. Neste trabalho, realizou-se um estudo comparativo entre métodos clássicos — como Insertion Sort, Selection Sort, Bubble Sort e Shell Sort — e algoritmos mais estruturados, como Merge Sort, Quick Sort e Heap Sort Min. Cada algoritmo foi examinado sob a perspectiva de seu funcionamento interno, pseudocódigo, custo computacional e desempenho prático em três cenários distintos: vetor crescente, decrescente e aleatório.

Além da análise teórica, foram coletados tempos reais de execução para grandes volumes de dados, permitindo observar como cada algoritmo responde a diferentes distribuições de entrada. Os resultados obtidos reforçam que algoritmos com complexidade assintótica mais eficiente, como Merge Sort e Quick Sort bem implementado, apresentam desempenho muito superior em comparação aos algoritmos quadráticos. Por outro lado, métodos simples como Insertion Sort e Shell Sort demonstraram ser altamente eficazes em situações específicas, especialmente quando a estrutura inicial dos dados favorece seu comportamento. Dessa forma, o estudo contribui para uma compreensão mais ampla sobre a aplicabilidade e eficiência dos principais algoritmos de ordenação.

Sumário

1	Resumo	1
2	Introdução	5
2.1	Fundamentação Teórica	5
3	Algoritmos	6
3.1	<i>Insertion Sort</i>	6
3.1.1	Pseudocódigo do <i>Insertion Sort</i>	6
3.1.2	Teste de Mesa do <i>Insertion Sort</i>	7
3.2	<i>Selection Sort</i>	7
3.2.1	Pseudocódigo do <i>Selection Sort</i>	8
3.2.2	Teste de Mesa do <i>Selection Sort</i>	9
3.3	<i>Shell Sort</i>	9
3.3.1	Pseudocódigo do <i>Shell Sort</i>	10
3.4	<i>Bubble Sort</i>	10
3.4.1	Pseudocódigo do <i>Bubble Sort</i>	11
3.4.2	Teste de Mesa do <i>Bubble Sort</i>	12
3.5	<i>Merge Sort</i>	12
3.5.1	Pseudocódigo do <i>Merge Sort</i>	13
3.5.2	Teste de Mesa do <i>Merge Sort</i>	14
3.6	<i>Quick Sort</i>	15
3.6.1	Pseudocódigo do <i>Quick Sort</i>	16
3.7	Heap Sort	16
3.7.1	Pseudocódigo do <i>Heap Sort Min</i>	17
4	Análise de Complexidade	18
4.1	<i>Insertion Sort</i>	18
4.1.1	Melhor Caso - <i>Insertion Sort</i>	18
4.1.2	Médio Caso - <i>Insertion Sort</i>	19
4.1.3	Pior Caso - <i>Insertion Sort</i>	21

4.2	<i>Selection Sort</i>	22
4.2.1	Melhor Caso - <i>Selection Sort</i>	22
4.2.2	Médio Caso - <i>Selection Sort</i>	23
4.2.3	Pior Caso - <i>Selection Sort</i>	24
4.3	<i>Shell Sort</i>	25
4.3.1	Funcionamento e Análise de Custos	25
4.3.2	Melhor Caso	26
4.3.3	Pior Caso	26
4.3.4	Caso Médio	26
4.3.5	Conclusão	26
4.4	<i>Bubble Sort</i>	27
4.4.1	Melhor Caso - <i>Selection Sort</i>	27
4.4.2	Médio Caso - <i>Bubble Sort</i>	28
4.4.3	Pior Caso - <i>Bubble Sort</i>	28
4.5	<i>Merge Sort</i>	29
4.5.1	Melhor Caso - <i>Merge Sort</i>	31
4.5.2	Médio Caso - <i>Merge Sort</i>	32
4.5.3	Pior Caso - <i>Merge Sort</i>	32
4.6	<i>Quick Sort</i>	32
4.6.1	Melhor Caso - <i>Quick Sort</i>	33
4.6.2	Médio Caso - <i>Quick Sort</i>	34
4.6.3	Pior Caso - <i>Quick Sort</i>	34
4.7	Heap Sort	34
4.7.1	Melhor Caso - <i>Heap Sort Min</i>	35
4.7.2	Médio Caso - <i>Heap Sort Min</i>	35
4.7.3	Pior Caso - <i>Heap Sort Min</i>	36
4.7.4	Complexidade da Função <i>BUILD-MIN-HEAP</i>	36
4.7.5	Complexidade da Função <i>MIN-HEAPIFY</i>	37
5	Tabela e Gráfico	38

5.1	<i>Insertion Sort</i>	38
5.2	<i>Selection Sort</i>	40
5.3	<i>Shell Sort</i>	42
5.4	<i>Bubble Sort</i>	44
5.5	<i>Merge Sort</i>	45
5.6	<i>Quick Sort</i>	47
5.6.1	<i>Quick Sort</i> — Pivô Primeiro Elemento	47
5.6.2	<i>Quick Sort</i> — Pivô Elemento Meio	49
5.6.3	<i>Quick Sort</i> — Pivô Aleatório	51
5.7	<i>Heap Sort Min</i>	53
6	Comparação Geral	55
6.1	Algoritmos Iniciais	55
6.2	Algoritmos Avançados	58
7	Conclusão	61
	Referências	63

2 Introdução

A ordenação é uma das operações fundamentais em ciência da computação, servindo de base para uma ampla gama de aplicações, desde bancos de dados até sistemas de recuperação de informação [2]. Embora algoritmos mais sofisticados como *Merge Sort* e *Quick Sort* sejam predominantes em sistemas de grande escala, algoritmos elementares como *Insertion Sort*, *Bubble Sort*, *Selection Sort* e *Shell Sort* desempenham um papel essencial no ensino e na compreensão inicial da análise de algoritmos [5].

Além de seu valor didático, esses algoritmos ainda encontram aplicações práticas em contextos específicos, como listas pequenas, quase ordenadas ou sistemas embarcados com restrições de memória [4]. O objetivo deste artigo é analisar esses algoritmos sob a perspectiva de suas características estruturais, complexidade e aplicabilidade, oferecendo também uma comparação entre eles.

2.1 Fundamentação Teórica

Um algoritmo de ordenação organiza elementos de uma sequência em ordem crescente ou decrescente de acordo com um critério de comparação [3]. A eficiência de um algoritmo é avaliada principalmente em termos de complexidade temporal e espacial, geralmente expressa na notação assintótica O , Ω e Θ [2].

Outros critérios relevantes incluem estabilidade — a preservação da ordem relativa entre elementos iguais — e adaptatividade, que se refere ao desempenho melhorado em entradas parcialmente ordenadas [1].

3 Algoritmos

3.1 *Insertion Sort*

O *Insertion Sort* é inspirado no processo manual de ordenar cartas. O algoritmo percorre o vetor da esquerda para a direita e, para cada elemento, insere-o na posição correta em relação aos elementos já analisados [5].

O procedimento funciona da seguinte forma: inicia-se com o segundo elemento, comparando-o com o primeiro e inserindo-o em ordem. Em seguida, o terceiro elemento é comparado com os anteriores, deslocando-os até que a posição correta seja encontrada. Esse processo é repetido até o último elemento da lista [2].

Por exemplo, ao ordenar a lista [5, 2, 4, 6, 1, 3], o *Insertion Sort* começará comparando o “2” com o “5”, inserindo-o antes. Em seguida, o “4” é comparado e posicionado entre o “2” e o “5”. O processo continua até que toda a lista esteja ordenada.

A complexidade do algoritmo é $O(n^2)$ no pior e no caso médio, mas se destaca no melhor caso, quando a lista já está ordenada, atingindo $O(n)$. Além disso, o *Insertion Sort* é estável e adaptativo, o que o torna muito eficiente em listas pequenas ou quase ordenadas [4].

3.1.1 Pseudocódigo do *Insertion Sort*

<i>INSERTION-SORT</i>		
CÓDIGO C	CUSTO	VEZES
1- for (i = 1; i < n; i++)	c_1	n
2- key = arr[i];	c_2	$n - 1$
3- j = i - 1;	c_3	$n - 1$
4- while (j >= 0 && arr[j] > key)	c_4	$\sum_{i=1}^{n-1} t_i$
5- arr[j + 1] = arr[j];	c_5	$\sum_{i=1}^{n-1} (t_i - 1)$
6- j = j - 1;	c_6	$\sum_{i=1}^{n-1} (t_i - 1)$
7- arr[j + 1] = key;	c_7	$n - 1$

Figura 1: Código C do *Insertion-Sort* com análise de custo e vezes.

3.1.2 Teste de Mesa do *Insertion Sort*

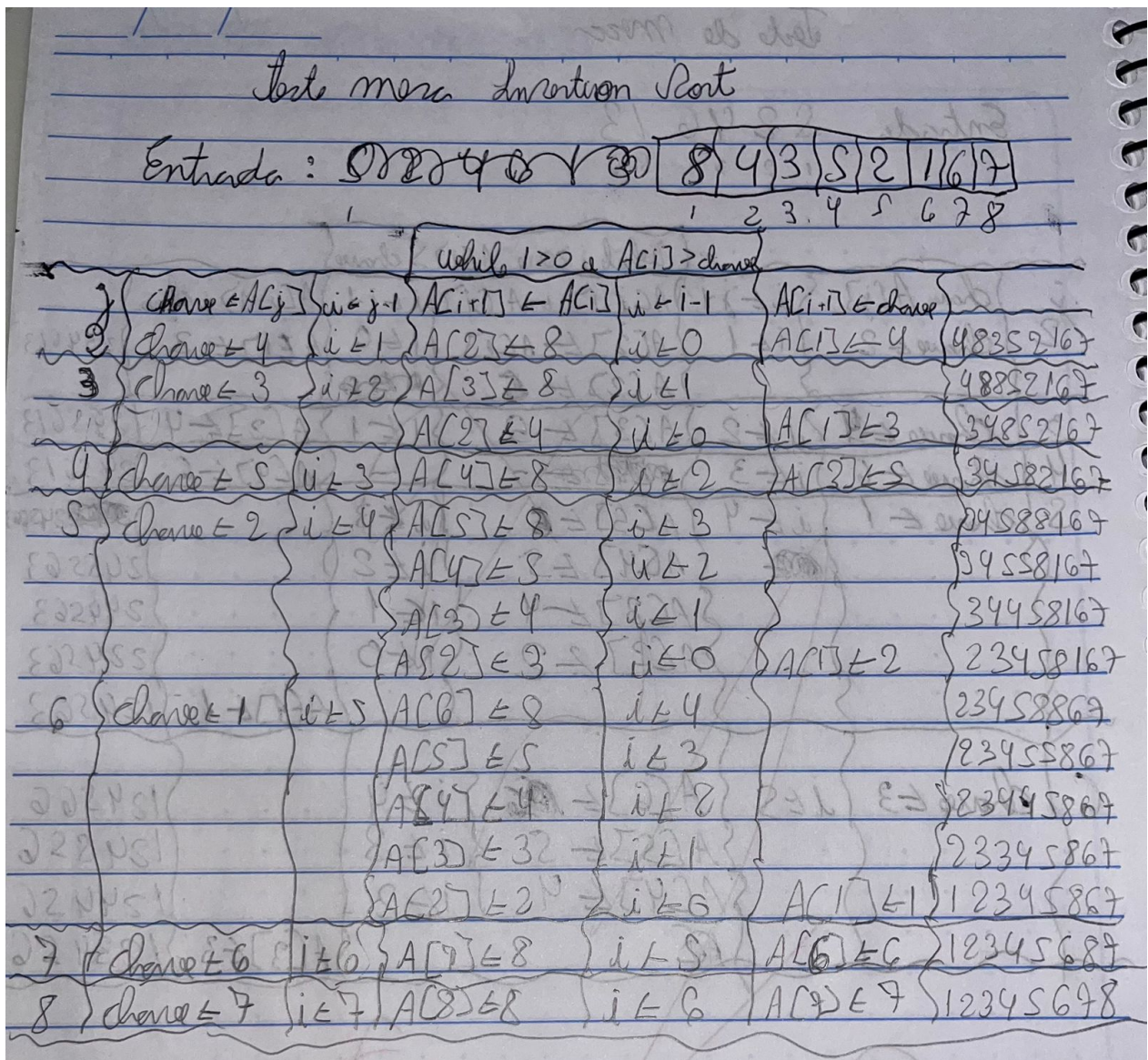


Figura 2: Teste de mesa do *Insertion Sort*.

3.2 Selection Sort

A *Selection Sort* baseia-se na ideia de selecionar o menor elemento da lista a cada iteração e colocá-lo na posição correta. Na primeira passagem, o menor elemento é encontrado e colocado na primeira posição. Na segunda passagem, o segundo menor é posicionado na segunda posição, e assim sucessivamente [5].

Por exemplo, na lista [29, 10, 14, 37, 13], o algoritmo identifica o “10” como o menor e troca-o com o primeiro elemento, resultando em [10, 29, 14, 37, 13]. Na segunda iteração, o “13” é o menor dos restantes, sendo colocado na segunda posição, e assim por diante até ordenar toda a lista.

Sua complexidade é $O(n^2)$ em todos os casos, já que sempre realiza $(n-1) + (n-2) + \dots + 1$ comparações, independentemente da ordem inicial [3]. Embora simples, não é estável em sua forma clássica e apresenta vantagem em cenários onde o número de trocas precisa ser minimizado [4].

3.2.1 Pseudocódigo do *Selection Sort*

<i>SELECTION-SORT</i>		
CÓDIGO C	CUSTO	VEZES
1- for (i = 0; i < n-1; i++)	c_1	n
2- min_idx = i;	c_2	$n - 1$
3- for (j = i+1; j < n; j++)	c_3	$\sum_{i=0}^{n-2} (n - i) = \frac{(n-1)(n+2)}{2}$
4- if (arr[j] < arr[min_idx])	c_4	$\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2}$
5- min_idx = j;	c_5	$\sum_{i=0}^{n-2} c_i$
6- // troca arr[i] com arr[min_idx]	c_6	$n - 1$

Figura 3: Código C do *Selection-Sort* com análise de custo e vezes.

3.2.2 Teste de Mesa do *Selection Sort*

Teste de mesa Selection Sort

$A = \{2, 5, 4, 1, 3\}$

i	$min = i$	$j = i+1$	$MIN = j$	$A[i] \leftrightarrow A[MIN]$	Array
1	$min = 1$	$j = 2$			2 5 4 1 3
		$j = 3$	$MIN = 3$	$A[i] = 4; A[MIN] = 5$	2 4 5 1 3
		$j = 4$	$MIN = 4$	$A[i] = 1; A[MIN] = 5$	2 4 1 5 3
		$j = 5$	$MIN = 5$	$A[i] = 3; A[MIN] = 5$	2 4 1 3 5
2	$MIN = 2$	$j = 3$	$MIN = 3$	$A[i] = 1; A[MIN] = 4$	2 4 4 3 5
		$j = 4$	$MIN = 4$	$A[i] = 3; A[MIN] = 4$	2 1 3 4 5
		$j = 5$			2 1 3 4 5
3	$MIN = 3$	$j = 4$			2 1 3 4 5
4	$MIN = 4$	$j = 5$			
5	$MIN = 1$	$j = 2$	$MIN = 2$	$A[i] = 1; A[MIN] = 2$	1 2 3 4 5

Figura 4: Teste de mesa do *Selection Sort*.

3.3 *Shell Sort*

O *Shell Sort*, proposto por Donald Shell em 1959, é considerado uma evolução do *Insertion Sort*. Ele introduz o conceito de “gaps” ou incrementos, permitindo comparar e ordenar elementos distantes entre si. A cada iteração, o valor do incremento é reduzido até que seja igual a 1, quando o algoritmo se torna equivalente ao *Insertion Sort* [3].

Por exemplo, com a lista [8, 5, 3, 7, 6, 2, 4, 1] e um incremento inicial de 4, o algoritmo compara os elementos nas posições (0,4), (1,5), (2,6), (3,7). Após ordenar essas

subseqüências, o incremento é reduzido para 2, e novas comparações são feitas em pares de elementos distantes duas posições, até que finalmente o incremento 1 finalize a ordenação.

A eficiência do *Shell Sort* depende da escolha da sequência de incrementos. Sequências como a de Hibbard ($2^k - 1$) ou de Pratt são estudadas por proporcionarem melhor desempenho. Embora não seja estável, pode atingir complexidade próxima de $O(n \log^2 n)$, o que o torna competitivo em listas médias [2].

3.3.1 Pseudocódigo do *Shell Sort*

<i>SHELL-SORT</i>		
CÓDIGO C	CUSTO	VEZES
1- for (int gap = n/2; gap > 0; gap /= 2)	c_1	$\approx \log n$
2- for (int i = gap; i < n; i++)	c_2	$\sum_{gap} (n - gap)$
3- int temp = arr[i];	c_3	$\sum_{gap} (n - gap)$
4- for (j=i; j>=gap && arr[j-gap]>temp; j-=gap)	c_4	$\sum_{gap} \sum_i t_{i,gap}$
5- arr[j] = arr[j - gap];	c_5	$\sum_{gap} \sum_i (t_{i,gap} - 1)$
6- arr[j] = temp;	c_6	$\sum_{gap} (n - gap)$

Figura 5: Código C do *Shell-Sort* com análise de custo e vezes.

3.4 *Bubble Sort*

O *Bubble Sort* é um dos algoritmos mais simples e consiste em percorrer repetidamente a lista, comparando pares adjacentes e trocando-os de lugar quando estão fora de ordem [3]. Esse processo se repete até que nenhuma troca seja necessária, indicando que a lista está ordenada.

Em termos de execução, considerando a lista [5, 1, 4, 2, 8], o algoritmo percorre comparando 5 e 1 (trocando), depois 5 e 4 (trocando), e assim por diante. Após a primeira iteração, o maior elemento já se encontra na última posição. Esse processo é repetido, com cada nova passagem colocando o próximo maior elemento em sua posição final.

Sua complexidade é $O(n^2)$ no caso médio e no pior caso, com melhor desempenho $O(n)$

quando a entrada já está ordenada, caso seja utilizada a versão otimizada que verifica se houve trocas. O *Bubble Sort* é estável, mas pouco eficiente, sendo empregado principalmente como recurso pedagógico [2].

3.4.1 Pseudocódigo do *Bubble Sort*

<i>BUBBLE-SORT</i>		
CÓDIGO C	CUSTO	VEZES
1- for (i = 0; i < n-1; i++)	c_1	n
2- for (j = 0; j < n-i-1; j++)	c_2	$\sum_{i=0}^{n-2} (n - i) = \frac{n(n+1)}{2} - 1$
3- if (arr[j] > arr[j+1])	c_3	$\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2}$
4- // troca arr[j] com arr[j+1]	c_4	T , onde $0 \leq T \leq \frac{n(n-1)}{2}$

Figura 6: Código C do *Bubble-Sort* com análise de custo e vezes.

3.4.2 Teste de Mesa do *Bubble Sort*

Teste de mesa Bubble Sort

$A = [5, 3, 8, 4, 6]$
 1 2 3 4 5

i	j	if $A[j] > A[j+1]$	$A[j] \leftrightarrow A[j+1]$	
1	1	5 > 3 ok	$A[j]=3; A[j+1]=5$	5 3 8 4 6
	2	5 > 8 FALSE		3 5 8 4 6
	3	8 > 4 ok	$A[j]=4; A[j+1]=8$	3 5 4 8 6
	4	8 > 6 ok	$A[j]=6; A[j+1]=8$	3 5 4 6 8
2	1	3 > 5 FALSE		
	2	5 > 4 ok	$A[j]=4; A[j+1]=5$	3 4 5 6 8 //
	3	5 > 6 FALSE		
3	1	3 > 4 FALSE		
	2	4 > 5 FALSE		
4	1	3 > 4 FALSE		

Figura 7: Teste de mesa do *Bubble Sort*.

3.5 *Merge Sort*

O *Merge Sort* é um algoritmo de ordenação baseado na estratégia *Dividir para Conquistar* (*Divide and Conquer*), proposto originalmente por John von Neumann em 1945 [2]. Seu funcionamento consiste em dividir recursivamente o vetor em duas metades até que cada subvetor contenha apenas um elemento, e então combinar (*merge*) essas partes de forma ordenada para reconstruir o vetor final.

O processo de ordenação pode ser descrito em três etapas principais: (1) divisão do vetor em duas partes aproximadamente iguais; (2) ordenação recursiva de cada uma dessas partes; e (3) intercalação ordenada dos elementos provenientes das duas metades. Essa abordagem

garante que, ao final de cada etapa de intercalação, o vetor parcial já esteja ordenado.

Por exemplo, ao ordenar a lista [38, 27, 43, 3, 9, 82, 10], o algoritmo a divide sucessivamente até obter listas unitárias: [38], [27], [43], [3], [9], [82], [10]. Em seguida, essas listas são combinadas e ordenadas em pares, formando [27, 38], [3, 43], [9, 82], [10]. O processo continua até resultar no vetor completamente ordenado [3, 9, 10, 27, 38, 43, 82].

O *Merge Sort* apresenta complexidade de tempo $O(n \log n)$ tanto no pior quanto no caso médio, tornando-o significativamente mais eficiente do que algoritmos quadráticos como o *Insertion Sort* ou o *Selection Sort*, especialmente para grandes conjuntos de dados. No entanto, seu principal inconveniente é o uso adicional de memória proporcional a n , necessária para armazenar os vetores auxiliares durante a etapa de intercalação. Além disso, o *Merge Sort* é um algoritmo estável, ou seja, preserva a ordem relativa de elementos iguais [4].

3.5.1 Pseudocódigo do *Merge Sort*

<i>MERGE-SORT</i>			
	CÓDIGO C	CUSTO	VEZES
1-	if (left < right)	c_1	$T(n)$
2-	mid = (left + right) / 2;	c_2	$T(n/2)$
3-	mergeSort(arr, left, mid);	c_3	$T(n/2)$
4-	mergeSort(arr, mid + 1, right);	c_4	$T(n/2)$
5-	merge(arr, left, mid, right);	c_5	$\Theta(n)$

Figura 8: Código C do *Merge Sort* com análise de custo e frequência.

3.5.2 Teste de Mesa do Merge Sort

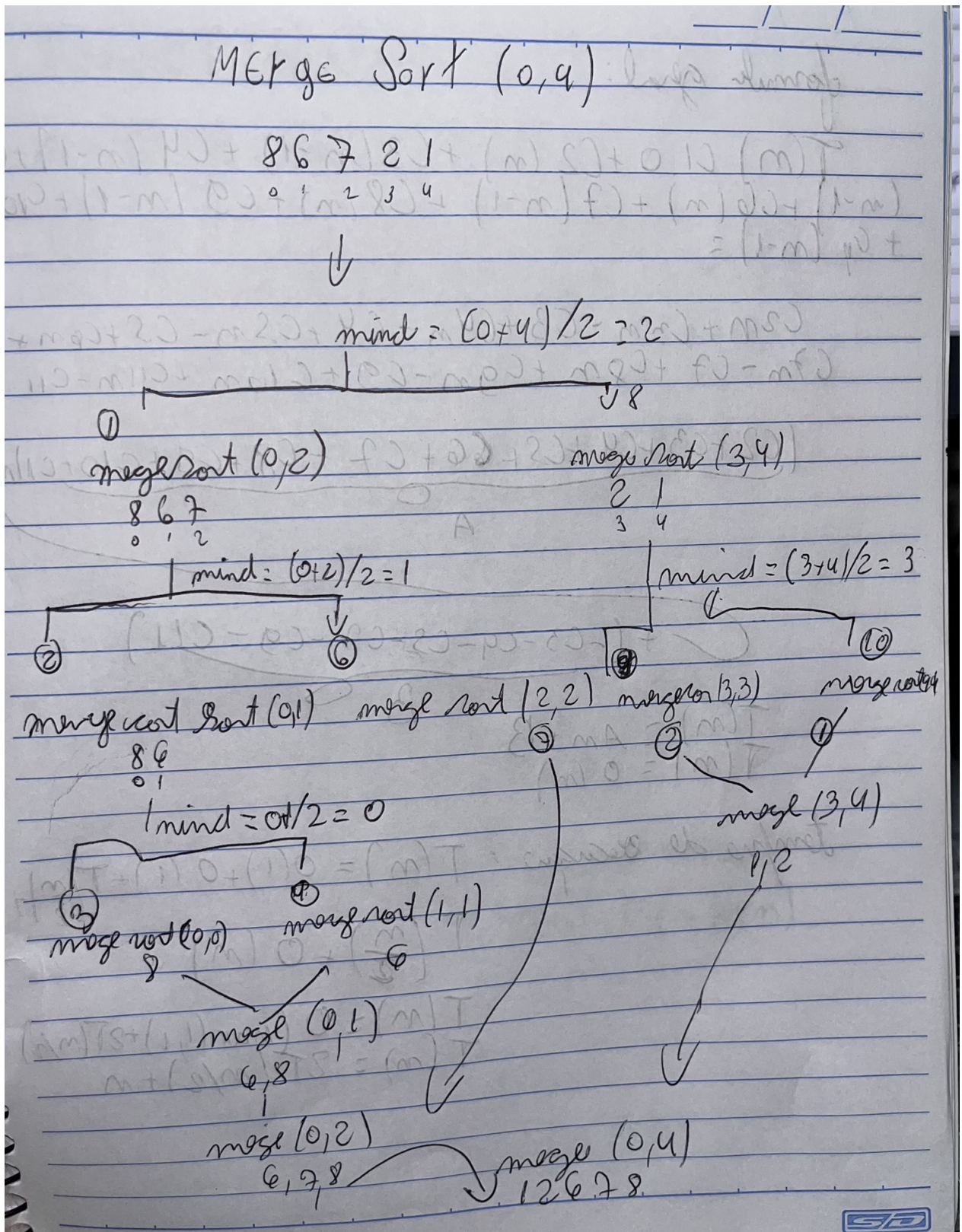


Figura 9: Teste de mesa do Merge Sort.

3.6 *Quick Sort*

O *Quick Sort* é um algoritmo de ordenação também baseado na estratégia *Dividir para Conquistar* (*Divide and Conquer*), proposto por Tony Hoare em 1960 [2]. É amplamente reconhecido pela sua eficiência prática e por ser um dos métodos de ordenação mais utilizados em sistemas reais devido ao seu bom desempenho médio e baixo uso de memória auxiliar.

O funcionamento do algoritmo pode ser descrito em três etapas principais: (1) seleção de um elemento chamado *pivô*; (2) particionamento do vetor em dois subvetores — um contendo os elementos menores que o pivô e outro com os maiores; e (3) aplicação recursiva do mesmo processo a cada um dos subvetores até que todo o vetor esteja ordenado. A ordenação completa ocorre quando as partições se reduzem a vetores unitários.

Por exemplo, ao ordenar a lista [10, 7, 8, 9, 1, 5], o *Quick Sort* pode escolher o último elemento (5) como pivô. O vetor é então particionado, resultando em [1] e [10, 7, 8, 9]. Em seguida, o algoritmo é chamado recursivamente sobre essas duas partes, repetindo o processo até que todas as sublistas estejam ordenadas, obtendo como resultado final [1, 5, 7, 8, 9, 10].

A eficiência do *Quick Sort* depende diretamente da escolha do pivô. Quando o pivô divide o vetor em partes aproximadamente iguais, o número de comparações é reduzido, garantindo complexidade de tempo $O(n \log n)$. Entretanto, quando o pivô é escolhido de forma desfavorável — como o menor ou maior elemento em um vetor já ordenado —, o número de comparações cresce para $O(n^2)$. Apesar disso, devido ao seu excelente desempenho médio e baixo uso de memória (pois opera in-place), o *Quick Sort* é frequentemente preferido em implementações práticas [4].

3.6.1 Pseudocódigo do *Quick Sort*

<i>QUICK-SORT</i>			
CÓDIGO C	CUSTO	VEZES	
1- if (low < high)	c_1	$T(n)$	
2- pivot = partition(arr, low, high);	c_2	$\Theta(n)$	
3- quickSort(arr, low, pivot - 1);	c_3	$T(k)$	
4- quickSort(arr, pivot + 1, high);	c_4	$T(n - k - 1)$	

Figura 10: Código C do *Quick Sort* com análise de custo e frequência.

3.7 Heap Sort

O *Heap Sort Min* é uma variação do algoritmo de ordenação baseado em estruturas de heap, que, ao contrário do *Heap Sort* tradicional (que utiliza um *max-heap*), emprega um *min-heap*. Nesse tipo de heap, o menor elemento sempre se encontra na raiz. O algoritmo funciona removendo repetidamente o menor elemento e reconstruindo o heap até que todos os itens tenham sido extraídos e armazenados em ordem crescente. A ideia central é manter a propriedade de heap a cada remoção do elemento raiz [2].

Por exemplo, ao ordenar a lista [8, 5, 3, 7, 6, 2, 4, 1], o algoritmo inicialmente constrói um *min-heap*. Após a construção, o menor elemento (1) é removido e colocado na primeira posição do vetor ordenado. Em seguida, o heap é reorganizado para restaurar a propriedade de *min-heap*, e o novo menor elemento é removido. Esse processo continua até que todos os elementos tenham sido extraídos em ordem crescente.

A eficiência do *Heap Sort Min* decorre da capacidade de construir o heap em tempo linear, $O(n)$, enquanto cada remoção da raiz seguida de reorganização custa $O(\log n)$. Como são feitas n remoções, a complexidade total do algoritmo é $O(n \log n)$. Além disso, o algoritmo não é estável, pois as operações de *heapify* podem alterar a ordem relativa de elementos iguais [4].

3.7.1 Pseudocódigo do *Heap Sort Min*

<i>HEAP-SORT-MIN</i>			
CÓDIGO C	CUSTO	VEZES	
1- buildMinHeap(arr, n);	c_1	$\Theta(n)$	
2- for (i = n-1; i >= 0; i--)	c_2	n	
3- swap(arr[0], arr[i]);	c_3	n	
4- minHeapify(arr, 0, i);	c_4	$n \log n$	
5- Função buildMinHeap(arr, n)			
6- for (i = n/2 - 1; i >= 0; i--)	c_5	$\Theta(n)$	
7- minHeapify(arr, i, n);	c_6	$\Theta(n)$	
8- Função minHeapify(arr, i, n)			
9- left = 2*i + 1;	c_7	$\approx n$	
10- right = 2*i + 2;	c_8	$\approx n$	
11- smallest = i;	c_9	$\approx n$	
12- if (left < n && arr[left] < arr[smallest])	c_{10}	n	
13- smallest = left;	c_{11}	n	
14- if (right < n && arr[right] < arr[smallest])	c_{12}	n	
15- smallest = right;	c_{13}	n	
16- if (smallest != i)	c_{14}	n	
17- swap(arr[i], arr[smallest]);	c_{15}	n	
18- minHeapify(arr, smallest, n);	c_{16}	$n \log n$	

Figura 11: Código C do *Heap Sort Min* com análise de custo e frequência.

4 Análise de Complexidade

4.1 *Insertion Sort*

A análise de complexidade de algoritmos é o estudo que busca avaliar a eficiência de um algoritmo em termos de recursos consumidos, principalmente o tempo de execução e o espaço de memória. No caso do *Insertion Sort*, o interesse está em medir quantas operações são realizadas para ordenar um vetor de tamanho n , considerando diferentes cenários de entrada. Essa análise é importante porque permite comparar o desempenho do *Insertion Sort* com outros algoritmos de ordenação e prever seu comportamento prático diante de diferentes distribuições de dados [2, 3, 5]. O funcionamento do algoritmo consiste em percorrer o vetor da esquerda para a direita e inserir cada novo elemento na posição correta em relação aos elementos anteriores, que já estão ordenados. Esse processo é realizado deslocando os elementos maiores até encontrar a posição adequada para a inserção.

4.1.1 Melhor Caso - *Insertion Sort*

O melhor caso ocorre quando o vetor de entrada já está ordenado em ordem crescente. Nesse cenário, o laço interno do algoritmo (responsável por deslocar elementos) não precisa realizar trocas, apenas uma comparação por elemento. Assim, o número de operações cresce linearmente com o tamanho do vetor, resultando em uma complexidade de tempo $O(n)$.

Melhor caso Insertion Sort

$$FG: C1 \cdot n + C2(n-1) + C3(n-1) + C4(n-1) + C5 \sum_{j=2}^n t_j + C6 \sum_{j=2}^n t_{j-1} + C7 \sum_{j=2}^n t_{j-1} + C8(n-1)$$

$$T(n) = C1 \cdot n + C2(n-1) + C3(n-1) + C4(n-1) + C5(n-1) + C8(n-1)$$

$$C1n + C2n - C2 + C3n - C3 + C4n - C4 + C5n - C5 + C8n - C8$$

$$(C1 + C2 + C3 + C4 + C5 + C8)n - (C2 + C3 + C4 + C5 + C8)$$

$$T(n) = A \cdot n - B$$

$$O(n)$$

Figura 12: Análise de complexidade para MELHOR CASO.

4.1.2 Médio Caso - *Insertion Sort*

O caso médio considera um vetor com elementos em ordem aleatória. Nessa situação, cada novo elemento percorre, em média, metade do subvetor já ordenado até encontrar sua posição correta. Assim, o número esperado de comparações é aproximadamente a metade do número obtido no pior caso, mas ainda assim o crescimento é quadrático. Portanto, a complexidade no caso médio é também $O(n^2)$.

Caso médio

$$T_j = \frac{j}{2}$$

$$T(n) = \left(\frac{2}{2}, \frac{3}{2}, \dots, \frac{n}{2} \right) \rightarrow \frac{1}{2} (2+3+\dots+n)$$

$$S_{pa} = \frac{n(a_1 + a_n)}{2} = \frac{(n-1)(n+2)}{2} = \frac{n^2 + n - 2}{4} \text{ ou}$$

$$\frac{n^2 + n - 1}{4} - \frac{1}{2}$$

$$\sum_{j=2}^n T_j - 1 \rightarrow \sum_{j=2}^n \frac{j-1}{2} = \frac{n^2 + n - 1}{4} - \frac{(n-1)}{2} = \frac{n^2 + n - 2 + 2n - 2}{4}$$

$$\frac{n^2 - n}{4}$$

$$T(n) = (C_1 + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5(\frac{n^2+n}{4} - \frac{1}{2}) + C_6(\frac{n^2+n}{4}) + C_7(\frac{n^2+n}{4}) + C_8(n-1))$$

$$C_1 n + C_2 n - C_2 + C_3 n - C_3 + C_4 n - C_4 + \frac{C_5 n^2}{4} + \frac{C_5 n}{4} - \frac{C_5}{2} + \frac{C_6 n^2}{4} - \frac{C_6 n}{4} + \frac{C_7 n^2}{4} - \frac{C_7 n}{4} + C_8 n - C_8$$

$$T(n) = \left(\frac{C_5 + C_6 + C_7}{4} \right) n^2 + \left(C_1 + C_2 + C_3 + \frac{C_5}{4} - \frac{C_6}{4} - \frac{C_7}{4} + C_8 \right) n + \left(-C_2 - C_3 - C_4 - \frac{C_5}{2} - C_8 \right)$$

$$T(n) = A n^2 + B n + C ; O(n^2)$$

Figura 13: Análise de complexidade para MÉDIO CASO.

4.1.3 Pior Caso - Insertion Sort

O pior caso acontece quando o vetor está em ordem decrescente. Nesse caso, cada elemento precisa ser deslocado até a primeira posição, gerando o maior número possível de comparações e trocas. Para o j -ésimo elemento, são necessárias $(j - 1)$ comparações. O custo total é proporcional à soma:

$$1 + 2 + 3 + \dots + (n - 1) = \frac{n(n - 1)}{2},$$

o que resulta em uma complexidade de tempo $O(n^2)$.

Pior Caso Insertion Sort

$$CS: \sum_{j=2}^n 1j = \sum_{j=2}^n j = \frac{(n-1)(2+n)}{2} = \frac{n^2 + n - 1}{2}$$

$$CB: \sum_{j=2}^n j - \sum_{j=2}^n 1 = \left(\frac{n^2 + n - 1}{2} \right) - (n-1) = \frac{n^2 - n}{2}$$

$$T(n) = C_1 \cdot n + C_2(n-1) + C_3(n-1) + C_4(n-1) + CS \left(\frac{n^2 + n - 1}{2} \right) +$$

$$CB \left(\frac{n^2 - n}{2} \right) + C_7 \left(\frac{n^2 - n}{2} \right) + C_8(n-1)$$

$$\left(\frac{CS + CB + C_7}{2} \right) n^2 + (C_1 + C_2 + C_3 + C_4 + CS + C_6 + C_7 + C_8)n -$$

$$(C_2 + C_3 + C_4 + C_5 + C_8)$$

$$T(n) \sim An^2 + Bn + C ;$$

$$O(n^2) //$$

Figura 14: Análise de complexidade para PIOR CASO.

4.2 *Selection Sort*

O *Selection Sort* é um algoritmo de ordenação simples que opera dividindo o vetor em duas partes: a sublista ordenada, que se expande da esquerda para a direita, e a sublista não ordenada, que diminui gradualmente. A cada iteração, o algoritmo seleciona o menor (ou maior) elemento da sublista não ordenada e o troca com o primeiro elemento dessa sublista. Esse processo é repetido até que todo o vetor esteja ordenado [2, 3, 5]. A análise de complexidade do *Selection Sort* está relacionada ao número de comparações e trocas realizadas, que são independentes da ordem inicial dos elementos, diferentemente do que ocorre no *Insertion Sort*.

4.2.1 Melhor Caso - *Selection Sort*

No melhor caso, quando o vetor já está ordenado em ordem crescente, o algoritmo ainda precisa comparar cada elemento com os demais para encontrar o mínimo. Portanto, o número de comparações não se altera. O que muda é o número de trocas: em um vetor já ordenado, apenas as trocas triviais (ou nenhuma, se o algoritmo for otimizado) são realizadas. Ainda assim, a complexidade de tempo é dominada pelas comparações, resultando em $O(n^2)$.

Melhor Caso Selection Sort

$$\sum_{i=1}^{n-1} (n-i) = \frac{n(1+(n-1))}{2} = \frac{n^2}{2}$$

$$T(n) = C_1(n-1) + C_2(n-1) + C_3\left(\frac{n^2}{2}\right) + C_4\left(\frac{n^2}{2}\right) + C_6(n-1)$$

$$T(n) = \left(\frac{C_3 + C_4}{2}\right)n^2 + (C_1 + C_2 + C_6)n - (C_1 + C_2 + C_6)$$

$$T(n) = An^2 + Bn - C$$

$$O(n^2) //$$

Figura 15: Análise de complexidade para MELHOR CASO.

4.2.2 Médio Caso - *Selection Sort*

No caso médio, considerando um vetor em ordem aleatória, o algoritmo continua realizando o mesmo padrão de comparações, independentemente da distribuição dos dados. A cada iteração i , são realizadas $(n - i)$ comparações, totalizando:

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2},$$

o que mantém a complexidade de tempo em $O(n^2)$.

Caso médio Selection Sort

$$C_5: \frac{n^2}{2} = \frac{n^2}{2} \cdot \frac{1}{2} = \frac{n^2}{4}$$

$$T(n) = C_1(n-1) + C_2(n-1) + C_3\left(\frac{n^2}{2}\right) + C_4\left(\frac{n^2}{2}\right) + C_5\left(\frac{n^2}{4}\right) + C_6(n-1)$$

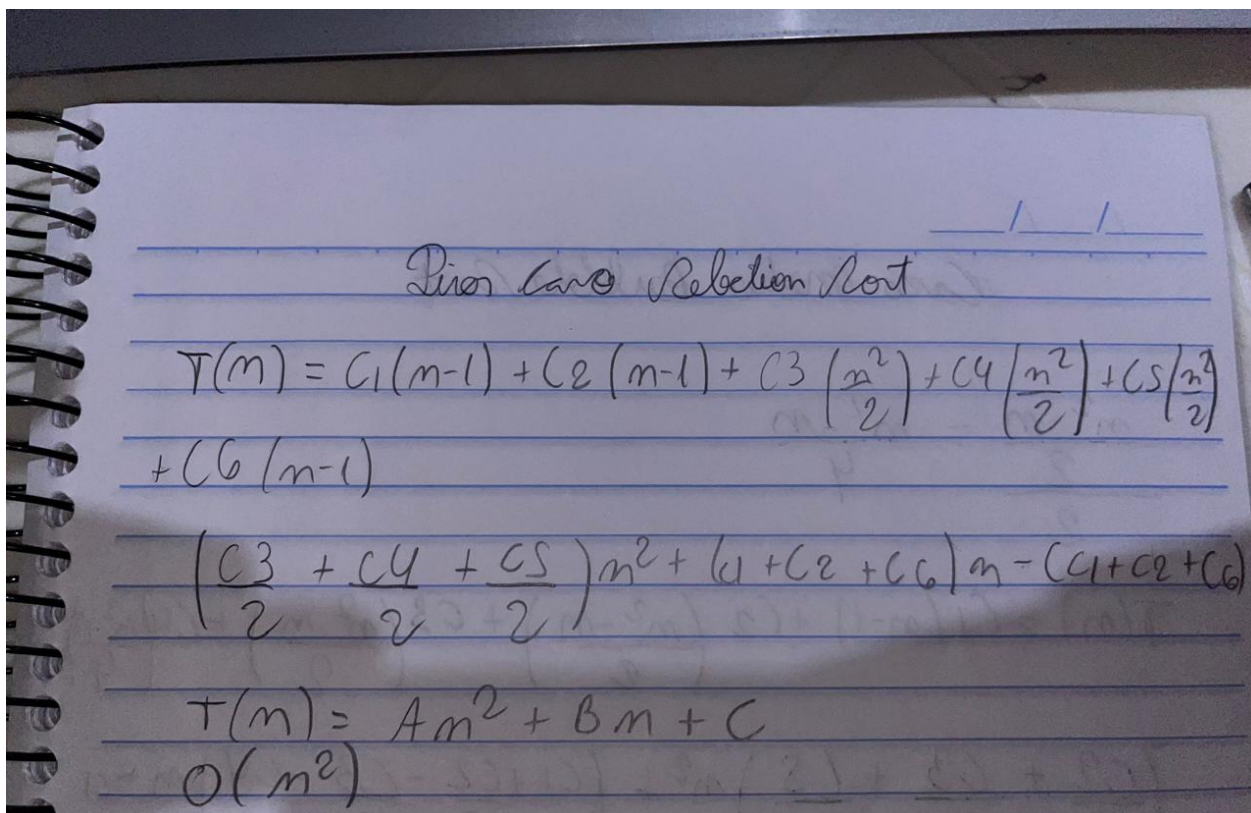
$$T(n) = \left(\frac{C_3}{2} + \frac{C_4}{2} + \frac{C_5}{4}\right)n^2 + (C_1 + C_2 + C_6)n - (C_1 + C_2 + C_6)$$

$$T(n) = An^2 + Bn - C, \quad O(n^2)$$

Figura 16: Análise de complexidade para MÉDIO CASO.

4.2.3 Pior Caso - Selection Sort

No pior caso, quando o vetor está em ordem decrescente, o algoritmo realiza o mesmo número de comparações, pois sempre precisa percorrer toda a sublista não ordenada em busca do menor elemento. O número de trocas pode aumentar, mas ainda assim permanece no máximo $n - 1$, o que é desprezível em relação ao número de comparações. Portanto, a complexidade de tempo no pior caso também é $O(n^2)$.



Pior Case Insertion Sort

$$T(n) = C_1(n-1) + C_2(n-1) + C_3\left(\frac{n^2}{2}\right) + C_4\left(\frac{n^2}{2}\right) + C_5\left(\frac{n^2}{2}\right) + C_6(n-1)$$

$$\left(\frac{C_3 + C_4 + C_5}{2}\right)n^2 + (C_1 + C_2 + C_6)n - (C_1 + C_2 + C_6)$$

$$T(n) = An^2 + Bn + C$$

$$O(n^2)$$

Figura 17: Análise de complexidade para PIOR CASO.

4.3 Shell Sort

O *Shell Sort*, proposto por Donald Shell em 1959, é uma generalização do *Insertion Sort*, criada para reduzir o custo das movimentações de elementos distantes [3, 2]. A ideia principal é ordenar inicialmente elementos que estão a uma certa distância (*gap*) uns dos outros, e gradualmente reduzir essa distância até que se faça uma ordenação final com o *Insertion Sort* tradicional.

4.3.1 Funcionamento e Análise de Custos

No *Shell Sort*, o vetor é dividido em subsequências formadas pelos elementos separados por um intervalo h , chamado de *incremento*. Cada subsequência é ordenada individualmente utilizando o *Insertion Sort*. Em seguida, o valor de h é reduzido e o processo se repete, até que $h = 1$, quando o vetor inteiro é ordenado.

A eficiência do algoritmo depende fortemente da sequência de incrementos escolhida. A sequência original proposta por Shell era $h = \lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \dots, 1$, mas outras sequências foram desenvolvidas posteriormente para melhorar o desempenho, como as sequências de Pratt, Sedgewick e Hibbard.

4.3.2 Melhor Caso

No melhor caso, o vetor já está ordenado ou próximo da ordem correta. Nesse cenário, cada subsequência já está em ordem, e o algoritmo apenas percorre os elementos, resultando em uma complexidade próxima a $O(n \log n)$, dependendo da sequência de incrementos.

4.3.3 Pior Caso

O pior caso do *Shell Sort* é mais difícil de analisar, pois depende da escolha da sequência de incrementos. Com a sequência original proposta por Shell, a complexidade no pior caso é $O(n^2)$. No entanto, com sequências mais eficientes, como a de Pratt ou de Sedgewick, o pior caso pode ser reduzido para $O(n^{3/2})$ ou até $O(n \log^2 n)$ [2].

4.3.4 Caso Médio

A análise do caso médio também é dependente da sequência escolhida. Usando sequências eficientes, o desempenho médio do algoritmo fica em torno de $O(n^{3/2})$. Estudos empíricos mostram que, embora não seja tão eficiente quanto algoritmos mais avançados como *Merge Sort* ou *Quick Sort*, o *Shell Sort* frequentemente apresenta bons resultados práticos para tamanhos moderados de entrada.

4.3.5 Conclusão

O *Shell Sort* representa uma evolução em relação ao *Insertion Sort*, reduzindo significativamente o número de movimentações necessárias para ordenar o vetor. Seu desempenho depende crucialmente da sequência de incrementos, variando entre $O(n \log n)$ e $O(n^2)$. Por esse motivo, é considerado um algoritmo de complexidade intermediária, com uso prático em cenários onde a simplicidade e o desempenho aceitável em entradas médias são suficientes.

A análise aqui apresentada é baseada em Knuth [3], Cormen et al. [2] e Ziviani [5]. “

4.4 *Bubble Sort*

O *Bubble Sort* é um dos algoritmos de ordenação mais simples. Seu funcionamento baseia-se em percorrer o vetor repetidas vezes, comparando pares de elementos adjacentes e trocando-os de lugar quando estão em ordem incorreta. A cada iteração, o maior elemento da parte não ordenada é "empurrado" para o final do vetor, como uma bolha que sobe à superfície [2, 3, 5]. A análise de complexidade desse algoritmo depende da ordem inicial dos elementos, pois em alguns casos ele pode finalizar antes ao detectar que não houve nenhuma troca em uma iteração.

4.4.1 Melhor Caso - *Selection Sort*

No melhor caso, quando o vetor já está ordenado, o algoritmo pode ser otimizado verificando se alguma troca foi realizada durante a varredura. Se nenhuma troca ocorrer, o algoritmo termina imediatamente após a primeira passagem. Assim, o número de comparações é $(n - 1)$ e não há trocas, resultando em complexidade de tempo linear: $O(n)$.

The image shows a handwritten derivation on lined paper for the best case complexity of Bubble Sort. The title is "Melhor Caso Bubble Sort". The derivation starts with the sum of comparisons: $\sum_{i=1}^{n-1} (n-i) = \frac{(n-1)(1+(n-1))}{2} = \frac{n^2-n}{2}$. Then, it defines the time function $T(n) = C_1(n-1) + C_2 \left(\frac{n^2-n}{2} \right) + C_3 \left(\frac{n^2-n}{2} \right)$. This is simplified to $T(n) = \left(\frac{C_2 + C_3}{2} \right) n^2 + \left(C_1 - \frac{C_2 + C_3}{2} \right) n + C_1$. Finally, it concludes with $T(n) = An^2 + Bn + C$ and $O(n^2)$.

$$\begin{aligned} \sum_{i=1}^{n-1} (n-i) &= \frac{(n-1)(1+(n-1))}{2} = \frac{n^2-n}{2} \\ T(n) &= C_1(n-1) + C_2 \left(\frac{n^2-n}{2} \right) + C_3 \left(\frac{n^2-n}{2} \right) \\ T(n) &= \left(\frac{C_2 + C_3}{2} \right) n^2 + \left(C_1 - \frac{C_2 + C_3}{2} \right) n + C_1 \\ T(n) &= An^2 + Bn + C \\ O(n^2) \end{aligned}$$

Figura 18: Análise de complexidade para MELHOR CASO.

4.4.2 Médio Caso - *Bubble Sort*

No caso médio, considerando os elementos dispostos aleatoriamente, o algoritmo executa, em média, metade do número máximo de trocas, mas o número de comparações continua sendo o mesmo que no pior caso, $\frac{n(n-1)}{2}$. Portanto, a complexidade média de tempo também é quadrática: $O(n^2)$.

The image shows a handwritten derivation on lined paper for the average case complexity of Bubble Sort. At the top, it is titled "Caso medio Bubble Sort". Below the title, the formula $\frac{n^2 - n}{2} = \frac{n^2 - n}{4}$ is written. Then, the time complexity function is defined as $T(n) = C_1(n-1) + C_2\left(\frac{n^2 - n}{2}\right) + C_3\left(\frac{n^2 - n}{2}\right) + C_4\left(\frac{n^2 - n}{4}\right)$. This is followed by the expansion of the terms: $\left(\frac{C_2}{2} + \frac{C_3}{2} + \frac{C_4}{2}\right)n^2 + \left(\frac{C_1}{2} + \frac{C_2}{2} - \frac{C_3}{2} - \frac{C_4}{2}\right)n + C_1$. Finally, it concludes with $T(n) = An^2 + Bn - C$ and $O(n^2)$.

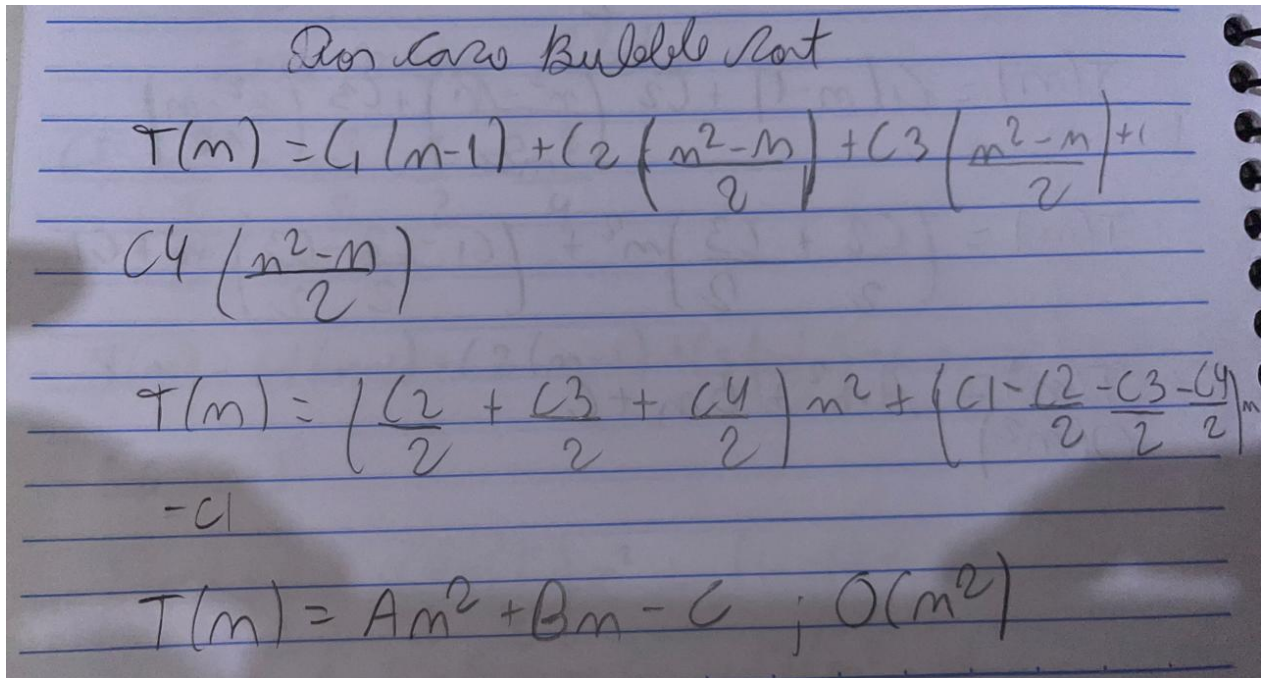
Figura 19: Análise de complexidade para MÉDIO CASO.

4.4.3 Pior Caso - *Bubble Sort*

No pior caso, quando o vetor está em ordem decrescente, o algoritmo precisa realizar o número máximo de comparações e trocas. Em cada uma das $(n - 1)$ iterações, percorrem-se os elementos restantes, realizando ao todo:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

comparações e praticamente o mesmo número de trocas. Isso resulta em uma complexidade quadrática: $O(n^2)$.



Pior Caso Bubble Sort

$$T(n) = C_1(n-1) + C_2 \left(\frac{n^2-n}{2} \right) + C_3 \left(\frac{n^2-n}{2} \right) + C_4 \left(\frac{n^2-n}{2} \right)$$

$$T(n) = \left(\frac{C_2}{2} + \frac{C_3}{2} + \frac{C_4}{2} \right) n^2 + \left(C_1 - \frac{C_2}{2} - \frac{C_3}{2} - \frac{C_4}{2} \right) n - C_1$$

$$T(n) = An^2 + Bn - C ; O(n^2)$$

Figura 20: Análise de complexidade para PIOR CASO.

4.5 Merge Sort

A análise de complexidade de algoritmos é fundamental para compreender o desempenho de diferentes métodos de ordenação em função do tamanho da entrada n . No caso do *Merge Sort*, o foco está em avaliar quantas operações são necessárias para dividir e mesclar os subvetores durante o processo de ordenação, considerando diferentes configurações iniciais dos dados. Esse tipo de análise é essencial para comparar sua eficiência com outros algoritmos, especialmente os de natureza quadrática, e prever o comportamento do algoritmo em cenários práticos [2, 3, 5]. O *Merge Sort* segue a estratégia de *Dividir para Conquistar*, onde o vetor é sucessivamente dividido em partes menores até que reste apenas um elemento, sendo então realizadas fusões ordenadas (*merge*) para reconstruir o vetor completamente ordenado.

fórmula geral:

$$T(n) = C1 \cdot 0 + C2(n) + C3(n-1) + C4(n-1) + C5(n-1) + C6(n) + C7(n-1) + C8(n) + C9(n-1) + C10(n) + C11(n-1) =$$

$$C2n + C3n - C3 + C4n - C4 + C5n - C5 + C6n + C7n - C7 + C8n + C9n - C9 + C10n + C11n - C11$$

$$\underbrace{(C2 + C3 + C4 + C5 + C6 + C7 + C8 + C9 + C10 + C11)}_A n$$

$$+ \underbrace{(-C3 - C4 - C5 - C7 - C9 - C11)}_B$$

$$T(n) = An + B$$

$$T(1) = O(1)$$

tempo de recursão: $T(n) = O(1) + O(1) + T\left(\frac{n}{2}\right) +$

$$T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = O(\max(1, 1) + 2T(n/2))$$

$$T(n) = 2T(n/2) + n$$

Figura 21: Fórmula Geral.

$$T(n) = 2T(n/2) + n$$

$$n = 2^k : k = \log_2 n ; T(2^0) = 1$$

$$T(2^k) = 2T(2^k/2) + 2^k = 2T(2^{k-1}) + 2^k$$

$$T(2^{k-1}) = 2T(2^{k-1}/2) + 2^{k-1} = 2T(2^{k-2}) + 2^{k-1}$$

$$2T(2^{k-1}) + 2^k$$

$$2[2T(2^{k-2}) + 2^{k-1}] + 2^k$$

$$2^2 T(2^{k-2}) + 2 \cdot 2^{k-1} + 2^k = 2^2 + (2 \cdot 2^{k-2}) + 2 \cdot 2^k$$

$$2^2 [2T(2^{k-3}) + 2^{k-2}] + 2 \cdot 2^{k-1} + 2^k$$

$$2^i T(2^{k-i}) + 2^k \cdot i$$

CP: $k-1=0$
 $i=k$

$$2^k \cdot 1 (2^k \cdot k)$$

$$2^k T(2^0) + 2^k \cdot k$$

$$2^k \cdot 1 (2^k \cdot k)$$

$$FF: T(n) = n + n \log_2 n$$

$$T(n) = O(n \log n)$$

Figura 22: Prova de complexidade.

4.5.1 Melhor Caso - Merge Sort

O melhor caso ocorre quando o vetor já está ordenado, o que, diferentemente de algoritmos como o *Insertion Sort*, não reduz significativamente o número de operações. Isso ocorre porque o *Merge Sort* realiza todas as divisões e fusões independentemente da ordem inicial

dos dados. Assim, mesmo que não sejam necessárias trocas, todas as comparações são feitas durante as etapas de intercalação. A complexidade de tempo no melhor caso é, portanto, $O(n \log n)$.

4.5.2 Médio Caso - *Merge Sort*

O caso médio considera que o vetor está em ordem aleatória. Nessa situação, o número de divisões e fusões é o mesmo do melhor caso, pois o algoritmo sempre executa as mesmas etapas de particionamento e combinação. A diferença está no número de comparações realizadas durante o processo de intercalação, que tende a variar levemente conforme a distribuição dos elementos. Ainda assim, o crescimento do tempo de execução segue a mesma ordem assintótica. Portanto, a complexidade no caso médio é também $O(n \log n)$.

4.5.3 Pior Caso - *Merge Sort*

O pior caso do *Merge Sort* ocorre quando o vetor de entrada está em ordem inversa. No entanto, assim como nos outros casos, o algoritmo precisa executar todas as divisões e fusões, sem variação significativa no número de operações. Dessa forma, o pior caso também possui complexidade de tempo $O(n \log n)$. O custo adicional pode surgir apenas em termos de espaço auxiliar, já que o algoritmo necessita de memória extra proporcional a n para armazenar os subvetores temporários durante o processo de intercalação.

4.6 *Quick Sort*

A análise de complexidade de algoritmos é essencial para compreender o desempenho do *Quick Sort* em diferentes situações, considerando o número de comparações e trocas realizadas em função do tamanho da entrada n . O *Quick Sort* utiliza a estratégia de *Dividir para Conquistar*, na qual um elemento do vetor é escolhido como pivô e os demais são particionados em dois subconjuntos: um com valores menores e outro com valores maiores que o pivô. O processo é então repetido recursivamente sobre cada subvetor até que toda a sequência esteja ordenada. Essa análise permite avaliar o impacto da escolha do pivô e a variação do desempenho do algoritmo conforme a configuração inicial dos dados [2, 3, 5].

Best Case Quick Sort V1

$$T(n) = T(n-1) + T(0) + O(n) \quad T(n) = T(n-1) + cn \quad (c = O(1))$$

$$T(n) = (T(n-2) + c(n-1) + cn) = T(n-3) + c(n-1) + cn \dots$$

$$T(n) = c \cdot (n + (n-1) + (n-2) + \dots + 1)$$

$$T(n) = c \cdot \sum_{i=1}^n i$$

$$T(n) = c \cdot \left(\frac{n(n+1)}{2} \right) = c \cdot \left(\frac{n^2 + n}{2} \right)$$

$$\left(\frac{c}{2} \right) n^2 + \left(\frac{c}{2} \right) n$$

$$A n^2 + B n + C \rightarrow O(n^2)$$

Figura 23: Análise de Complexidade.

4.6.1 Melhor Caso - Quick Sort

O melhor caso ocorre quando o pivô escolhido em cada etapa divide o vetor em duas partes de tamanhos aproximadamente iguais. Nesse cenário, a profundidade da recursão é mínima, e o número de comparações realizadas é proporcional a $n \log n$. Isso acontece, por exemplo, quando os elementos estão distribuídos de forma que o pivô sempre se encontra próximo do valor mediano. Assim, o tempo de execução cresce de forma quase linear-logarítmica, resultando em uma complexidade de tempo $O(n \log n)$.

4.6.2 Médio Caso - *Quick Sort*

No caso médio, considera-se que os pivôs são escolhidos aleatoriamente e que, em média, as partições não são perfeitamente balanceadas, mas ainda mantêm tamanhos razoavelmente próximos. Nessa situação, o número de comparações cresce proporcionalmente a $n \log n$, embora com uma constante de tempo ligeiramente maior que no melhor caso devido ao desbalanceamento parcial das divisões. Dessa forma, a complexidade no caso médio é também $O(n \log n)$, sendo o motivo pelo qual o *Quick Sort* é amplamente considerado um dos algoritmos de ordenação mais eficientes na prática.

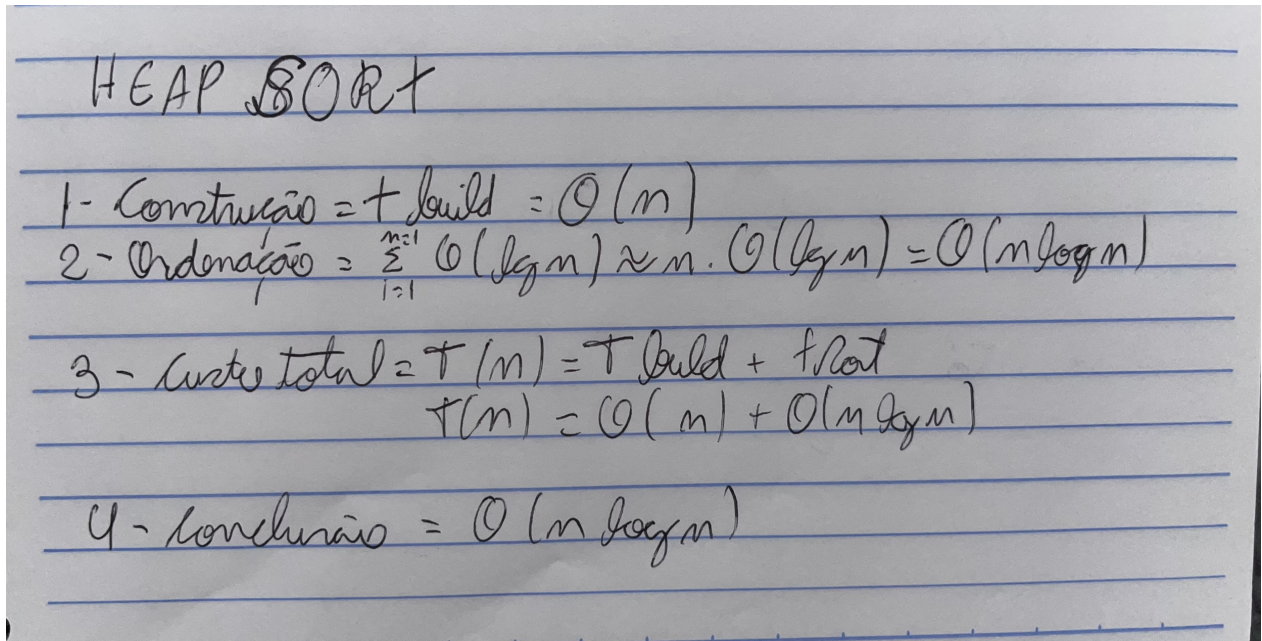
4.6.3 Pior Caso - *Quick Sort*

O pior caso ocorre quando o pivô escolhido é o menor ou o maior elemento do vetor, o que faz com que uma das partições fique vazia e a outra contenha quase todos os elementos restantes. Essa situação é comum quando o vetor já está ordenado ou em ordem inversa e o pivô é sempre escolhido de forma determinística (por exemplo, o primeiro ou o último elemento). Nesse caso, o número de comparações e trocas cresce quadraticamente, e a complexidade de tempo atinge $O(n^2)$. Apesar disso, em implementações modernas, o uso de pivôs aleatórios ou estratégias como o *median-of-three* minimiza significativamente a ocorrência desse cenário.

4.7 Heap Sort

A análise de complexidade de algoritmos é fundamental para compreender o desempenho do *Heap Sort* sob diferentes condições de entrada, especialmente no que diz respeito ao número de comparações, movimentações e chamadas às funções do heap em função do tamanho da entrada n . O *Heap Sort* baseia-se na construção e manutenção de uma estrutura de dados chamada *heap*, que é uma forma especial de árvore binária completa que obedece à propriedade de heap. No caso do *Min-Heap*, cada nó é menor que seus filhos, garantindo que o menor elemento esteja sempre na raiz. O processo de ordenação consiste em construir o heap a partir do vetor original e, em seguida, extrair repetidamente o menor elemento, reorganizando a estrutura a cada remoção. Essa abordagem garante um desempenho previsível e estável, independentemente da configuração inicial dos dados, já que a ordem dos elementos

não altera significativamente o número de operações exigidas pelo algoritmo [2, 3, 5].



HEAP SORT

1- Construção = $T_{\text{build}} = O(n)$

2- Ordenação = $\sum_{i=1}^{n-1} O(\log n) \approx n \cdot O(\log n) = O(n \log n)$

3- Custo total = $T(n) = T_{\text{build}} + T_{\text{sort}}$
 $T(n) = O(n) + O(n \log n)$

4- Conclusão = $O(n \log n)$

Figura 24: Análise de complexidade para *HEAP-SORT*

4.7.1 Melhor Caso - *Heap Sort Min*

No melhor caso, a estrutura inicial já está muito próxima de um *min-heap* válido, o que reduz o número de correções realizadas pela função `minHeapify`. Apesar disso, o algoritmo ainda realiza a construção completa do heap e todas as remoções necessárias, mantendo sua complexidade assintótica em $O(n \log n)$.

4.7.2 Médio Caso - *Heap Sort Min*

O caso médio assume que o vetor está em ordem aleatória. A construção do heap continua custando $\Theta(n)$ e a reestruturação após cada remoção mantém o custo de $O(\log n)$. Assim, a complexidade permanece $O(n \log n)$, com pequenas variações constantes dependendo da distribuição dos elementos.

4.7.3 Pior Caso - *Heap Sort Min*

O pior caso ocorre quando cada remoção da raiz causa uma cascata de trocas profundas no heap, forçando a função `minHeapify` a percorrer todo o caminho até as folhas. Mesmo nesse cenário, o custo total continua sendo $O(n \log n)$, já que o algoritmo realiza exatamente n reestruturações, cada uma limitada pela altura do heap ($\log n$).

4.7.4 Complexidade da Função *BUILD-MIN-HEAP*

A função *BUILD-MIN-HEAP* é responsável por transformar um vetor não ordenado em uma estrutura de *Min-Heap*. O processo consiste em aplicar a função *MIN-HEAPIFY* de maneira decrescente, iniciando no último nó não folheado (posição $\lfloor n/2 \rfloor$) até a raiz do heap. Embora à primeira vista pareça que o custo seria proporcional a $O(n \log n)$, devido à chamada do *MIN-HEAPIFY* em cada um dos nós internos, a análise mais detalhada mostra que os nós próximos às folhas têm altura muito baixa, reduzindo drasticamente o custo total. Dessa forma, a soma dos custos ponderados pela altura dos nós resulta em complexidade linear. Assim, o tempo total da função é $O(n)$, o que a torna extremamente eficiente para preparar a estrutura antes da fase de ordenação [2, 3].

Handwritten analysis of the complexity of the BUILD-MIN-HEAP function:

- 1 - Número de nós por altura = $\lfloor n/2^{h+1} \rfloor$
- 2 - Custo total = $\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\log n} \frac{h}{2^h}\right)$
- 3 - $T(n) = O(n \cdot 2) = O(n)$

Figura 25: Análise de complexidade para *BUILD-MIN-HEAP*

4.7.5 Complexidade da Função *MIN-HEAPIFY*

A função *MIN-HEAPIFY* tem como objetivo restaurar a propriedade de *Min-Heap* em uma subárvore cuja raiz pode estar violando essa propriedade. O algoritmo compara o nó atual com seus filhos e, caso necessário, realiza uma troca seguida de uma chamada recursiva para o filho que recebeu o valor deslocado. O tempo de execução depende da altura da subárvore, pois o elemento pode precisar descer até um dos níveis mais profundos do heap. Como um heap binário completo possui altura $\log n$, o custo máximo de cada chamada ao *MIN-HEAPIFY* é $O(\log n)$. Esse comportamento ocorre independentemente da configuração inicial dos dados, pois a altura da árvore determina diretamente o número máximo de comparações e possíveis trocas. Portanto, a complexidade de tempo da função *MIN-HEAPIFY* é $O(\log n)$ no pior caso, mantendo-se dentro da mesma ordem de grandeza para os demais cenários [2, 5].

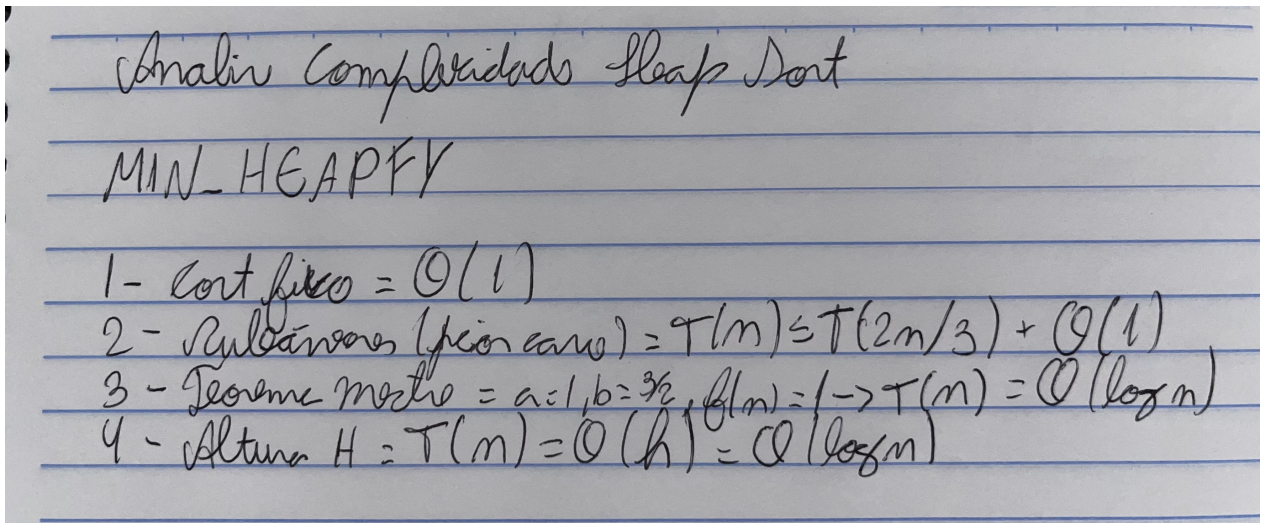


Figura 26: Análise de complexidade para *MIN-HEAPIFY*

5 Tabela e Gráfico

5.1 *Insertion Sort*

Tabela do Algoritmo InsertionSort

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000003	0.000004	0.000015	0.00014	0.001735	0.008208
Decrescente (s)	0.000004	0.000047	0.003734	0.157777	14.835865	3353.120071
Randômico (s)	0.000004	0.000027	0.002371	0.10117	7.100071	1452.086733

Figura 27: Tabela dos tempos de execução do *insertion-sort*.

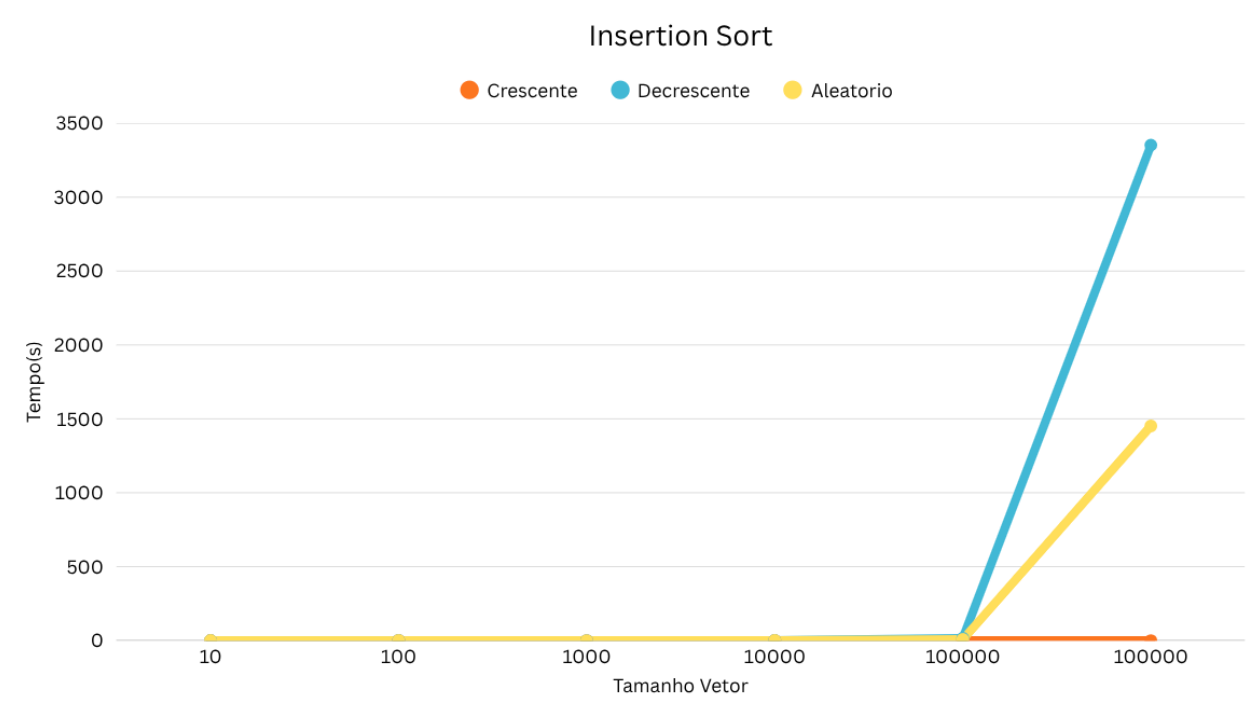


Figura 28: Crescimento dos tempos de execução do *insertion-sort*.

A Tabela 27 e o Gráfico 28 apresentam o desempenho do algoritmo *Insertion Sort* em diferentes cenários de ordenação: vetor crescente, decrescente e aleatório.

Caso crescente (melhor caso): Os resultados mostram que o tempo de execução cresce de forma quase linear, permanecendo bastante baixo mesmo para vetores muito grandes (por exemplo, aproximadamente 0,008 segundos para um vetor de 1.000.000 elementos). Isso ocorre porque, nesse cenário, o algoritmo realiza apenas comparações, sem a necessidade de movimentar elementos, caracterizando a complexidade de melhor caso $O(n)$.

Caso decrescente (pior caso): Observa-se um crescimento extremamente acentuado no tempo de execução, chegando a mais de 3353 segundos (quase uma hora) para 1.000.000 elementos. Esse resultado está de acordo com a complexidade de pior caso $O(n^2)$, já que cada inserção exige o deslocamento de todos os elementos previamente ordenados.

Caso aleatório (médio caso): Os tempos de execução também seguem um crescimento quadrático, mas com valores intermediários entre o melhor e o pior caso. Para um vetor de 1.000.000 elementos, o tempo de execução foi de aproximadamente 1452 segundos (cerca de 24 minutos), refletindo o comportamento esperado do caso médio $O(n^2)$.

Análise geral: O gráfico evidencia que a curva do caso crescente é praticamente imperceptível devido aos valores muito baixos, enquanto os casos decrescente e aleatório apresentam crescimento explosivo a partir de vetores com 100.000 elementos. Dessa forma, conclui-se que o *Insertion Sort* é eficiente apenas para vetores pequenos ou já quase ordenados, tornando-se inviável para grandes volumes de dados desordenados. “

5.2 *Selection Sort*

Tabela do Algoritmo SelectionSort

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000004	0.000036	0.002835	0.158725	12.849113	2159.365736
Decrescente (s)	0.000005	0.00004	0.001972	0.166381	13.614106	2286.440144
Randômico (s)	0.000002	0.000025	0.003602	0.163217	12.86936	2136.469251

Figura 29: Tabela dos tempos de execução do *Selection Sort*.

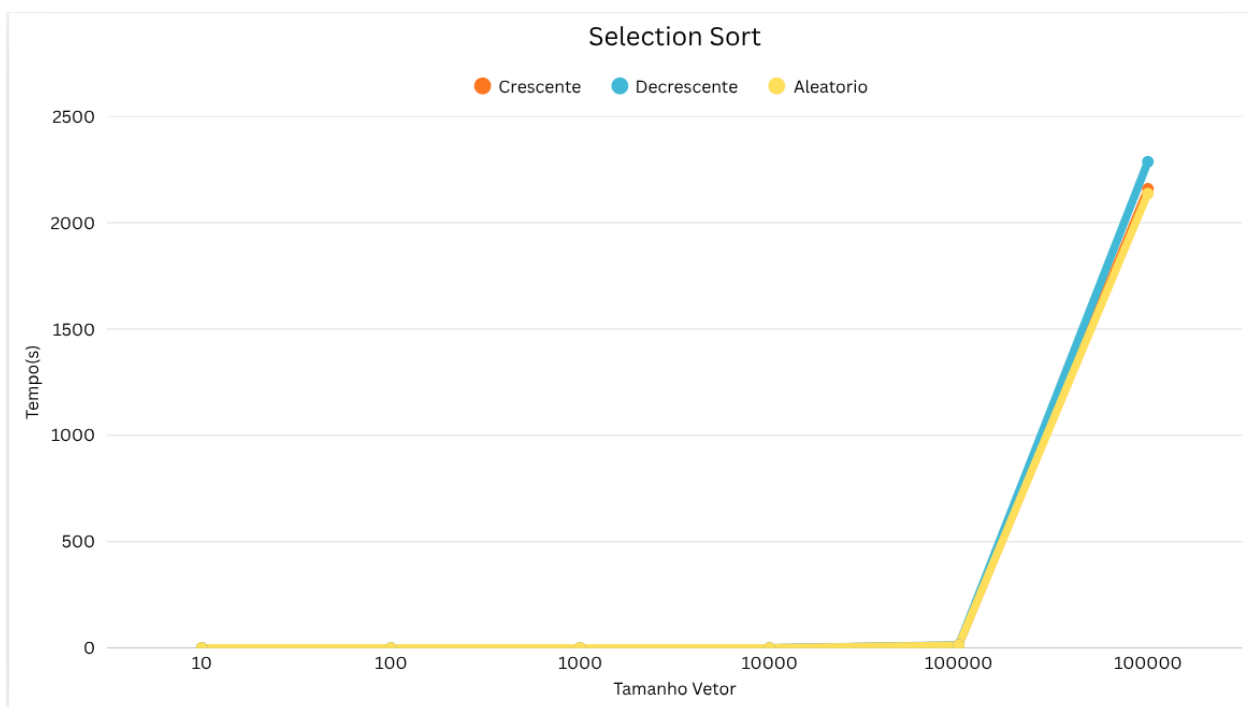


Figura 30: Crescimento dos tempos de execução do *Selection Sort*.

A Tabela 29 e o Gráfico 30 apresentam os resultados de desempenho do algoritmo *Selection Sort* em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O tempo de execução aumenta significativamente à medida que o tamanho do vetor cresce, alcançando aproximadamente 2159 segundos para 1.000.000 elementos. Diferente do *Insertion Sort*, o desempenho do *Selection Sort* não é impactado de forma relevante pela ordenação inicial, uma vez que o algoritmo percorre todo o vetor em busca do menor elemento a cada iteração, resultando em complexidade $O(n^2)$ independentemente do caso.

Caso decrescente: Os resultados mostram tempos bastante próximos ao caso crescente, chegando a cerca de 2286 segundos para 1.000.000 elementos. Esse comportamento reforça o fato de que o algoritmo executa o mesmo número de comparações e trocas, independentemente da disposição inicial dos dados.

Caso aleatório: De maneira semelhante, o desempenho segue a mesma tendência quadrática, com tempo final próximo de 2136 segundos para 1.000.000 elementos. As variações entre os cenários são mínimas, reforçando a característica determinística do *Selection Sort* em termos de número de operações.

Análise geral: O gráfico mostra que as três curvas (crescente, decrescente e aleatório) praticamente se sobrepõem, evidenciando que o *Selection Sort* possui comportamento uniforme em diferentes distribuições dos dados. Isso ocorre porque o algoritmo realiza sempre o mesmo número de comparações ($\frac{n^2}{2}$ em média), sendo pouco sensível à ordem inicial do vetor. Dessa forma, sua aplicação é limitada a vetores pequenos, pois o tempo de execução cresce rapidamente em função do tamanho da entrada.

5.3 *Shell Sort*

Tabela do Algoritmo ShellSort

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000002	0.000006	0.000067	0.000972	0.012071	0.152342
Decrescente (s)	0.000002	0.000009	0.000095	0.001361	0.017654	0.203638
Randômico (s)	0.000002	0.000015	0.000195	0.003232	0.045729	0.639742

Figura 31: Tabela dos tempos de execução do *Shell Sort*.

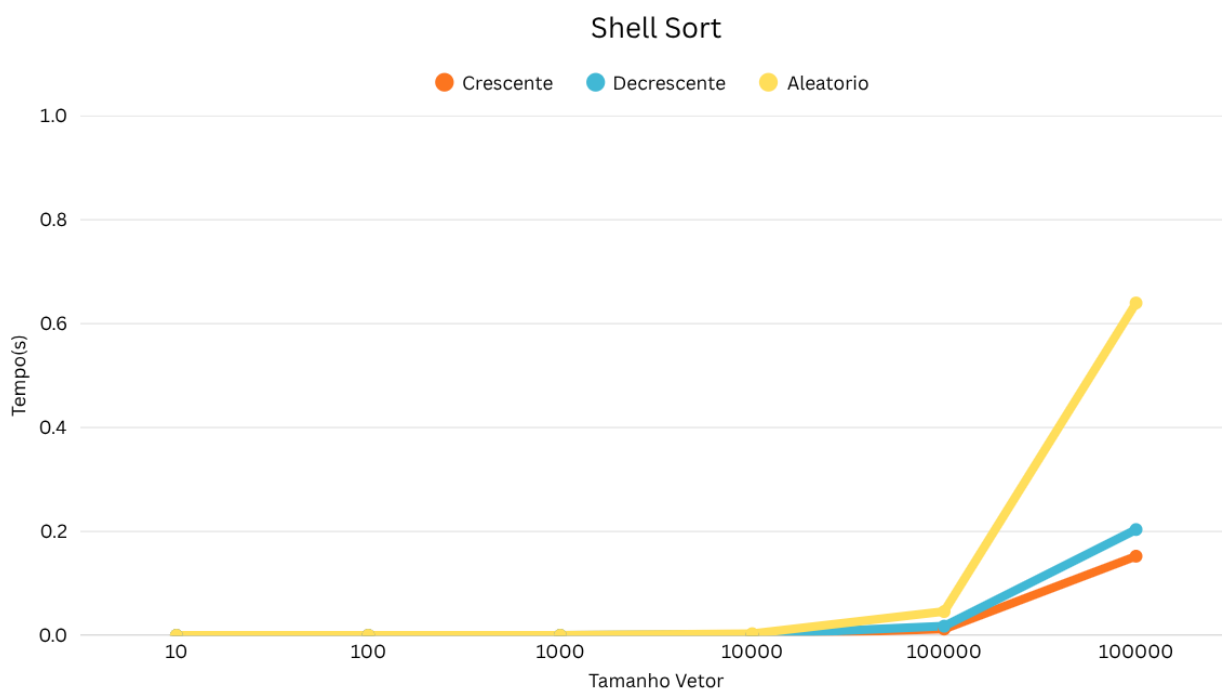


Figura 32: Crescimento dos tempos de execução do *Shell Sort*.

A Tabela 31 e o Gráfico 32 apresentam os resultados de desempenho do algoritmo *Shell Sort* em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O desempenho do algoritmo é bastante eficiente, mesmo para grandes volumes de dados. Para 1.000.000 de elementos, o tempo de execução foi de aproximadamente 0.15 segundos, destacando-se como muito inferior aos resultados obtidos com algoritmos quadráticos como *Insertion Sort* e *Selection Sort*. Isso se deve à redução drástica do número de comparações e movimentações proporcionada pela estratégia de intervalos (*gaps*).

Caso decrescente: O tempo de execução foi ligeiramente superior ao do caso crescente, alcançando cerca de 0.20 segundos para 1.000.000 elementos. Essa diferença ocorre porque o vetor em ordem inversa tende a exigir mais movimentações em algumas fases do algoritmo, mas ainda assim o crescimento permanece bastante controlado.

Caso aleatório: Apresenta desempenho intermediário, mas com tempo superior aos outros dois cenários, chegando a cerca de 0.63 segundos para 1.000.000 elementos. Isso ocorre porque a distribuição aleatória tende a exigir mais reordenações nos diferentes intervalos, embora o crescimento continue assintoticamente menor que o dos algoritmos puramente quadráticos.

Análise geral: O *Shell Sort* mostra-se significativamente mais eficiente que algoritmos quadráticos simples, apresentando tempos de execução na ordem de milissegundos até mesmo para vetores muito grandes. Além disso, nota-se que a diferença entre os cenários crescente, decrescente e aleatório é perceptível, mas não altera substancialmente a eficiência global do algoritmo. Essa característica decorre de sua complexidade média subquadrática, que varia dependendo da escolha da sequência de incrementos, podendo se aproximar de $O(n^{3/2})$ ou até mesmo de $O(n \log^2 n)$ em versões otimizadas.

5.4 *Bubble Sort*

Tabela do Algoritmo BubbleSort

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000003	0.000046	0.003333	0.169886	13.55703	2330.192283
Decrescente (s)	0.000004	0.000264	0.007026	0.286469	25.413987	3861.928359
Randômico (s)	0.000004	0.000069	0.005253	0.258482	29.621962	4808.911548

Figura 33: Tabela dos tempos de execução do *Bubble Sort*.

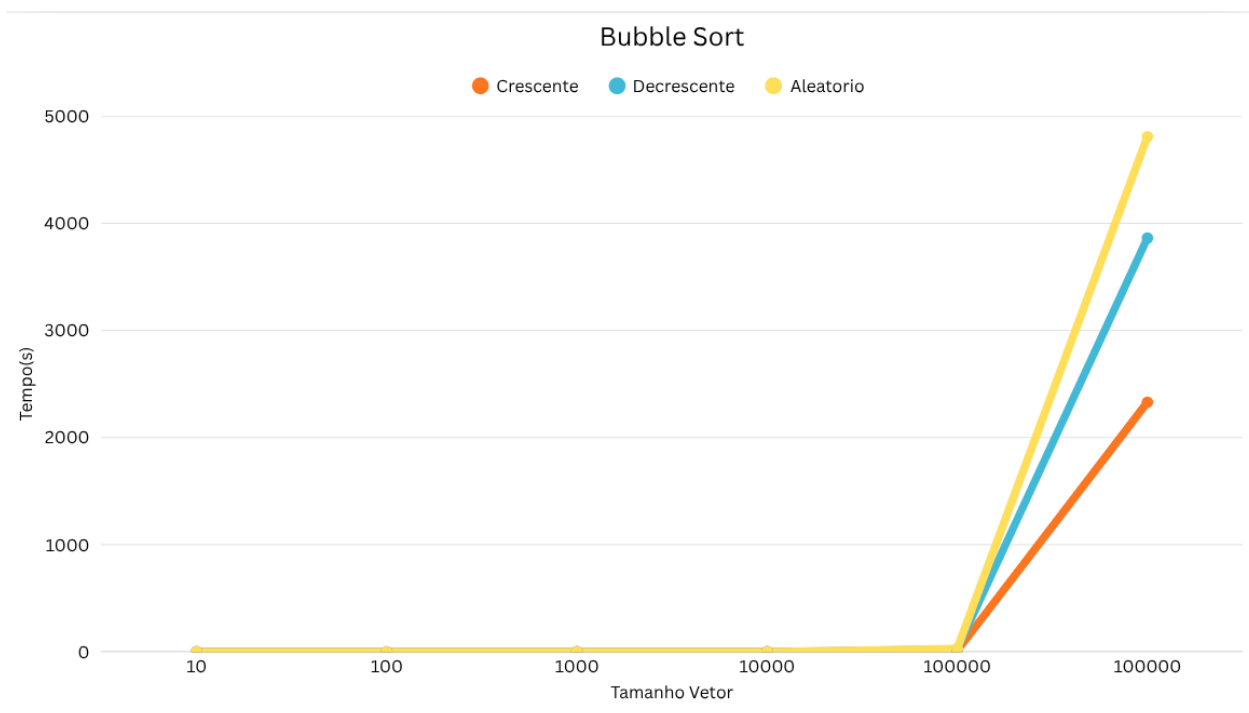


Figura 34: Crescimento dos tempos de execução do *Bubble Sort*.

A Tabela 33 e o Gráfico 34 apresentam os resultados de desempenho do algoritmo *Bubble Sort* em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O *Bubble Sort* apresenta um tempo ligeiramente inferior aos outros cenários, pois, em um vetor já ordenado, o número de trocas é reduzido. No entanto, mesmo nesse caso mais favorável, para 1.000.000 elementos o tempo de execução alcança mais de 2.300 segundos, refletindo sua complexidade quadrática $O(n^2)$.

Caso decrescente: Este cenário é um dos piores para o algoritmo, já que exige o máximo de trocas possíveis. O tempo de execução cresce rapidamente, chegando a mais de 3.800 segundos para 1.000.000 elementos, demonstrando a ineficiência do *Bubble Sort* em vetores ordenados de forma inversa.

Caso aleatório: Apresenta o pior desempenho entre os três cenários, com tempo de execução acima de 4.800 segundos para 1.000.000 elementos. Isso ocorre porque, em média, o algoritmo realiza muitas comparações e trocas desnecessárias, refletindo sua baixa eficiência em dados não estruturados.

Análise geral: O *Bubble Sort* confirma sua reputação como um dos algoritmos de ordenação menos eficientes em termos práticos, especialmente para grandes conjuntos de dados. Embora seja didaticamente útil para introduzir conceitos básicos de ordenação, sua complexidade quadrática inviabiliza seu uso em aplicações reais. A diferença entre os três cenários é visível, mas em todos os casos o crescimento do tempo é exponencial em comparação a algoritmos mais eficientes, como o *Shell Sort* ou o *Quick Sort*.

5.5 *Merge Sort*

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000003	0.000013	0.000139	0.001456	0.009037	0.070331
Decrescente (s)	0.000003	0.000012	0.000112	0.001455	0.012480	0.066560
Randômico (s)	0.000004	0.000017	0.000221	0.002707	0.014115	0.150550

Figura 35: Tabela dos tempos de execução do *Merge Sort*.

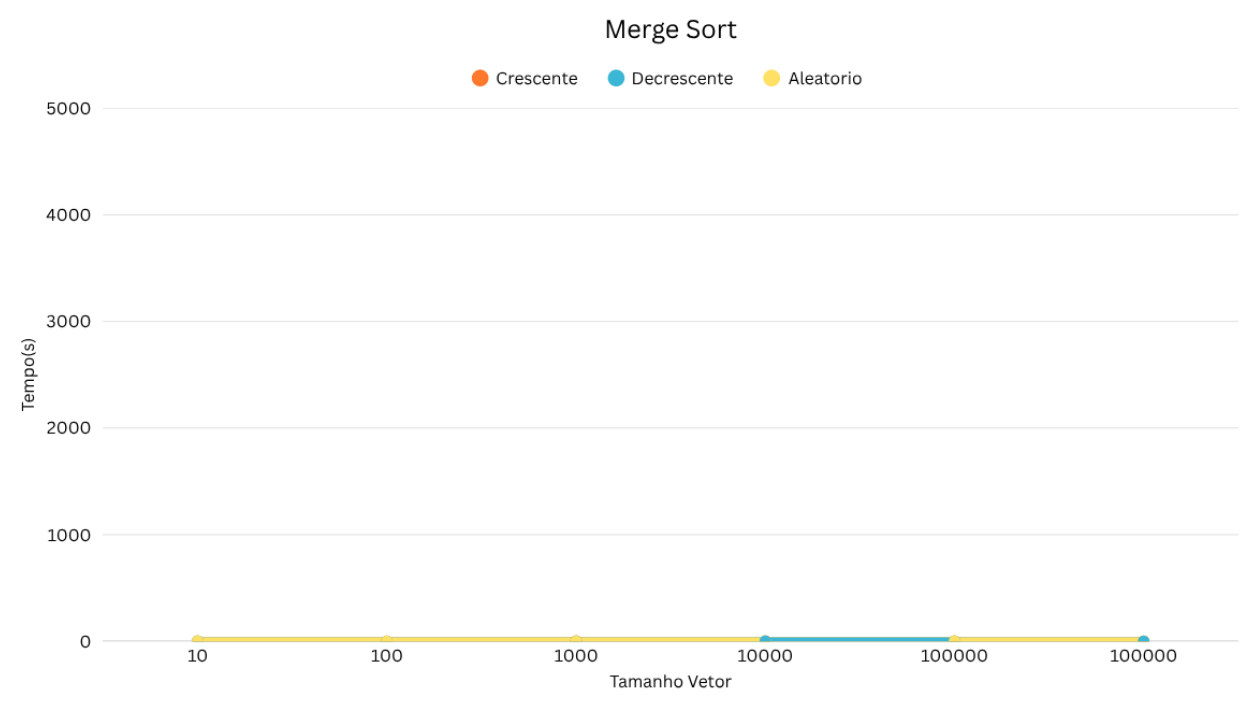


Figura 36: Crescimento dos tempos de execução do *Merge Sort*.

A Tabela 35 e o Gráfico 36 apresentam os resultados de desempenho do algoritmo *Merge Sort* em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O desempenho do *Merge Sort* mostrou-se altamente eficiente e consistente, refletindo a natureza estável e previsível do algoritmo. Para 1.000.000 de elementos, o tempo de execução foi de aproximadamente 0.070 segundos, um valor significativamente inferior aos tempos observados nos algoritmos quadráticos, como *Bubble Sort*, *Insertion Sort* e *Selection Sort*. Esse resultado evidencia a eficiência da estratégia de divisão e conquista empregada pelo *Merge Sort*, que reduz o número de comparações e movimentações ao dividir o vetor em partes menores e ordená-las de forma recursiva.

Caso decrescente: O tempo de execução manteve-se muito próximo ao do caso crescente, com cerca de 0.066 segundos para 1.000.000 de elementos. Isso ocorre porque, independentemente da disposição inicial dos dados, o *Merge Sort* executa sempre o mesmo número de divisões e mesclagens, apresentando comportamento estável e previsível em qualquer cenário. Assim, sua complexidade é praticamente inalterada pela ordem de entrada, o que o diferencia dos algoritmos baseados em comparações diretas entre elementos adjacentes.

Caso aleatório: Nesse cenário, o tempo de execução foi ligeiramente superior, alcançando aproximadamente 0.150 segundos para 1.000.000 de elementos. Essa diferença é esperada devido ao maior custo nas operações de mesclagem de subvetores mais desbalanceados, comuns em conjuntos de dados aleatórios. Ainda assim, o desempenho permaneceu dentro da faixa de eficiência esperada, comprovando a robustez do algoritmo frente a diferentes distribuições de entrada.

Análise geral: De modo geral, o *Merge Sort* apresenta um comportamento assintótico muito superior ao dos algoritmos quadráticos, com complexidade de tempo $O(n \log n)$ em todos os casos — melhor, médio e pior. Essa estabilidade se reflete nos resultados experimentais, onde as variações entre os cenários são pequenas, reforçando a previsibilidade e eficiência do método. Embora requeira espaço adicional proporcional a n para a etapa de mesclagem, seu custo computacional permanece controlado mesmo para grandes volumes de dados. Dessa forma, o *Merge Sort* destaca-se como um dos algoritmos de ordenação mais eficientes e consistentes da literatura clássica, conforme discutido em [2, 3, 5].

5.6 *Quick Sort*

5.6.1 *Quick Sort* — Pivô Primeiro Elemento

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000002	0.000016	0.001159	0.070670	3.234820	-
Decrescente (s)	0.000003	0.000023	0.001390	0.095368	7.778974	-
Randômico (s)	0.000003	0.000024	0.000167	0.001626	0.012657	0.119124

Figura 37: Tabela dos tempos de execução do *Quick Sort* (Pivô Primeiro Elemento).

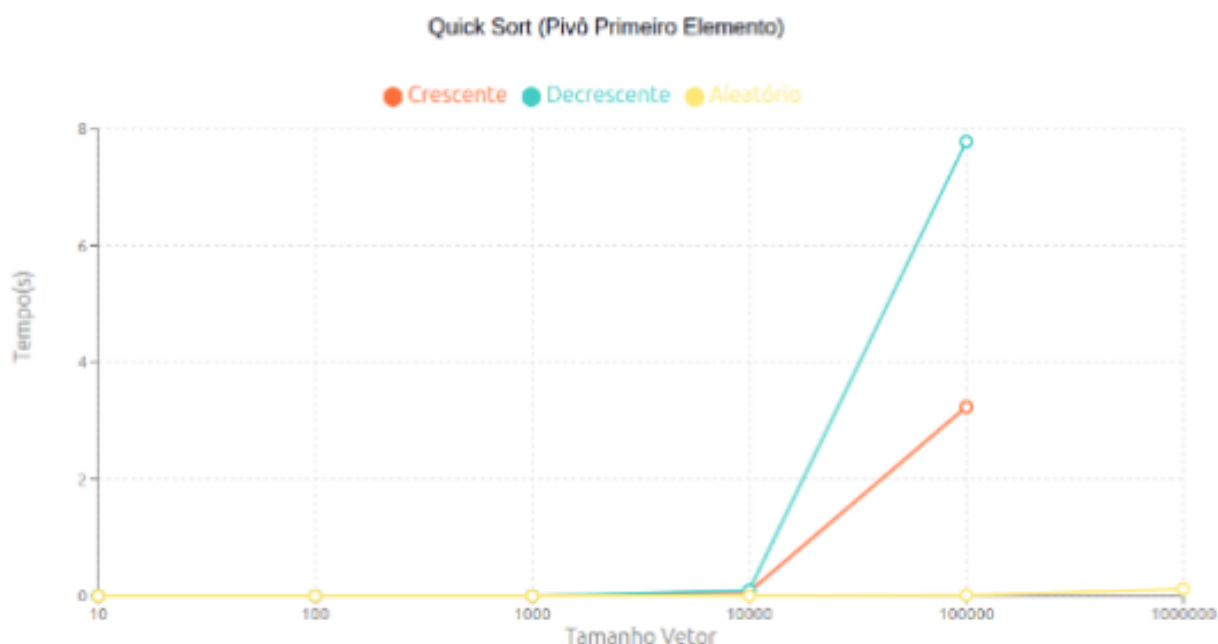


Figura 38: Crescimento dos tempos de execução do *Quick Sort* (Pivô Primeiro Elemento).

A Tabela 37 e o Gráfico 38 apresentam os resultados de desempenho do algoritmo *Quick Sort* utilizando o primeiro elemento como pivô, em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O *Quick Sort* com pivô no primeiro elemento apresentou desempenho crítico neste cenário, evidenciando o pior caso do algoritmo. Para vetores pequenos, o tempo de execução foi aceitável (0.000002 segundos para 10 elementos), porém o crescimento tornou-se exponencial com o aumento do tamanho do vetor. Para 100.000 elementos, o tempo alcançou aproximadamente 3.235 segundos, representando uma degradação severa de desempenho. Ao tentar processar 1.000.000 de elementos, o algoritmo resultou em erro de *core dump* devido a estouro de pilha (*stack overflow*). Esse comportamento ocorre porque, em um vetor já ordenado, escolher sempre o primeiro elemento como pivô leva à criação de partições extremamente desbalanceadas, gerando uma recursão de profundidade $O(n)$, o que excede o limite da pilha de execução e degrada a complexidade temporal para $O(n^2)$.

Caso decrescente: Similar ao caso crescente, o cenário decrescente também induziu o pior caso do *Quick Sort* com pivô no primeiro elemento. Os tempos de execução foram ligeiramente superiores, atingindo aproximadamente 7.779 segundos para 100.000 elementos.

Novamente, ao processar 1.000.000 de elementos, ocorreu erro de *core dump* por estouro de pilha. A escolha inadequada do pivô em um vetor inversamente ordenado resulta no mesmo problema de partições desbalanceadas, onde cada chamada recursiva processa apenas um elemento a menos que a anterior, gerando n níveis de recursão e causando tanto o estouro de pilha quanto a complexidade quadrática.

Caso aleatório: Neste cenário, o *Quick Sort* demonstrou seu comportamento esperado no caso médio, apresentando desempenho significativamente superior. Para 1.000.000 de elementos, o tempo de execução foi de apenas 0.119 segundos, valor comparável ao *Merge Sort* e ordens de magnitude mais rápido que os casos crescente e decrescente. Isso ocorre porque, em dados aleatórios, a probabilidade de escolher pivôs que gerem partições balanceadas é maior, resultando em profundidade de recursão $O(\log n)$ e complexidade temporal $O(n \log n)$.

5.6.2 *Quick Sort* — Pivô Elemento Meio

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000001	0.000003	0.000050	0.000571	0.007400	0.034494
Decrescente (s)	0.000003	0.000010	0.000068	0.000992	0.007500	0.043463
Randômico (s)	0.000002	0.000007	0.000064	0.000860	0.010609	0.117348

Figura 39: Tabela dos tempos de execução do *Quick Sort* (Pivô Elemento Meio).

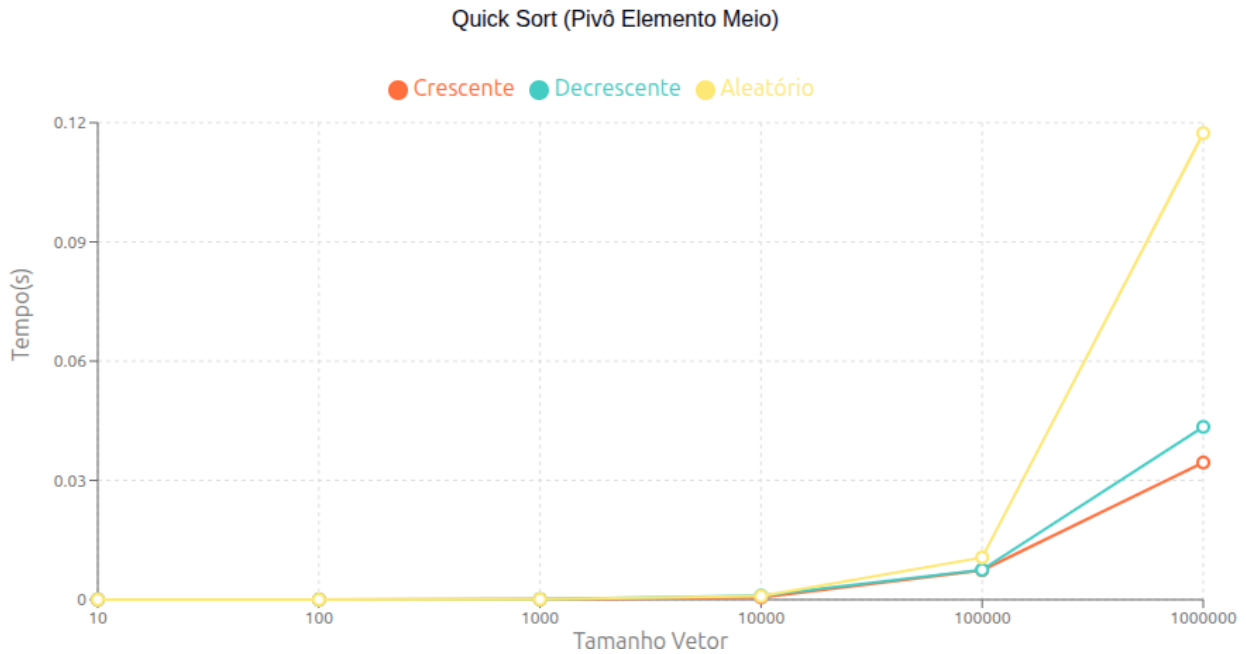


Figura 40: Crescimento dos tempos de execução do *Quick Sort* (Pivô Elemento Meio).

A Tabela 39 e o Gráfico 40 apresentam os resultados de desempenho do algoritmo *Quick Sort* utilizando o elemento do meio como pivô.

Caso crescente: Com a escolha do elemento do meio como pivô, o *Quick Sort* apresentou desempenho excelente no cenário crescente. Para 1.000.000 de elementos, o tempo de execução foi de apenas 0.034 segundos, demonstrando melhoria dramática em relação à versão com pivô no primeiro elemento. Essa estratégia garante partições mais balanceadas em vetores ordenados, evitando o pior caso e mantendo a complexidade próxima de $O(n \log n)$. A profundidade de recursão permaneceu controlada, eliminando o risco de estouro de pilha.

Caso decrescente: O desempenho manteve-se igualmente eficiente, com tempo de execução de aproximadamente 0.043 segundos para 1.000.000 de elementos. A escolha do pivô no meio do vetor novamente resultou em partições balanceadas, demonstrando que esta estratégia é eficaz tanto para vetores crescentes quanto decrescentes. A diferença mínima entre os tempos confirma a estabilidade do algoritmo frente a diferentes ordenações iniciais.

Caso aleatório: No cenário aleatório, o tempo de execução foi de aproximadamente 0.117 segundos para 1.000.000 de elementos, ligeiramente superior aos casos ordenados. Embora o pivô no meio não garanta partições perfeitamente balanceadas em dados aleatórios, o

desempenho permaneceu dentro do esperado para a complexidade $O(n \log n)$, demonstrando robustez do algoritmo mesmo em condições menos favoráveis.

5.6.3 *Quick Sort* — Pivô Aleatório

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000003	0.000015	0.000108	0.001097	0.009365	0.065026
Decrescente (s)	0.000004	0.000015	0.000117	0.001407	0.010779	0.067089
Randômico (s)	0.000003	0.000018	0.000155	0.002153	0.014077	0.123560

Figura 41: Tabela dos tempos de execução do *Quick Sort* (Pivô Aleatório).

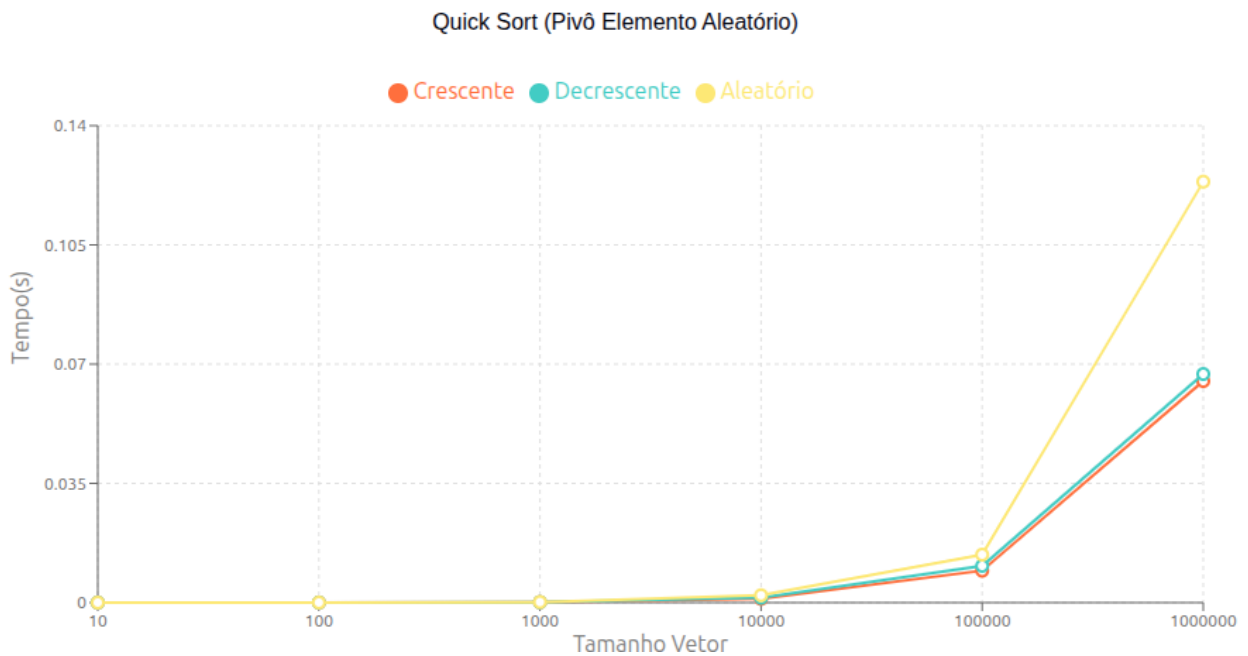


Figura 42: Crescimento dos tempos de execução do *Quick Sort* (Pivô Aleatório).

A Tabela 41 e o Gráfico 42 apresentam os resultados de desempenho do algoritmo *Quick Sort* utilizando um elemento aleatório como pivô.

Caso crescente: A estratégia de pivô aleatório apresentou desempenho excelente no caso crescente, com tempo de execução de aproximadamente 0.065 segundos para 1.000.000

de elementos. A aleatoriedade na escolha do pivô elimina a dependência da ordem inicial dos dados, garantindo partições balanceadas em média e evitando o pior caso sistemático observado na versão com pivô fixo no primeiro elemento. O tempo foi ligeiramente superior à versão com pivô no meio, mas manteve-se dentro da complexidade $O(n \log n)$.

Caso decrescente: O desempenho no caso decrescente foi similar ao crescente, com tempo de aproximadamente 0.067 segundos para 1.000.000 de elementos. A pequena variação entre os cenários ordenados confirma que a aleatoriedade do pivô torna o algoritmo insensível à ordem inicial dos dados, distribuindo uniformemente a probabilidade de partições balanceadas independentemente da configuração de entrada.

Caso aleatório: No cenário aleatório, o tempo de execução foi de aproximadamente 0.124 segundos para 1.000.000 de elementos, o maior entre os três casos. Isso ocorre porque, mesmo com pivô aleatório, a natureza já desordenada dos dados pode ocasionalmente resultar em partições menos balanceadas. No entanto, o desempenho permaneceu consistente com a complexidade esperada $O(n \log n)$, demonstrando que a aleatorização é uma estratégia robusta para garantir bom desempenho médio.

Análise comparativa: Os resultados experimentais evidenciam a importância crítica da estratégia de escolha do pivô no *Quick Sort*. Enquanto o pivô fixo no primeiro elemento degrada catastroficamente para $O(n^2)$ em vetores ordenados, causando inclusive estouro de pilha, as estratégias de pivô no meio e pivô aleatório mantêm desempenho consistente próximo de $O(n \log n)$ em todos os cenários. O pivô no meio apresentou os melhores tempos absolutos para dados ordenados (0.034-0.043 segundos), enquanto o pivô aleatório oferece garantias probabilísticas de bom desempenho independente da entrada. Ambas as estratégias aprimoradas demonstram superioridade sobre o pivô fixo e competitividade com o *Merge Sort*, confirmando o *Quick Sort* como um dos algoritmos de ordenação mais eficientes quando implementado com escolha inteligente de pivô, conforme amplamente discutido na literatura [2].

5.7 *Heap Sort Min*

TIPO	10	100	1.000	10.000	100.000	1.000.000
Crescente (s)	0.000004	0.000016	0.000198	0.002941	0.026204	0.176194
Decrescente (s)	0.000004	0.000016	0.000155	0.002809	0.033141	0.179830
Randômico (s)	0.000003	0.000018	0.000302	0.003400	0.035326	0.258076

Figura 43: Tabela dos tempos de execução do *Heap Sort Min*.

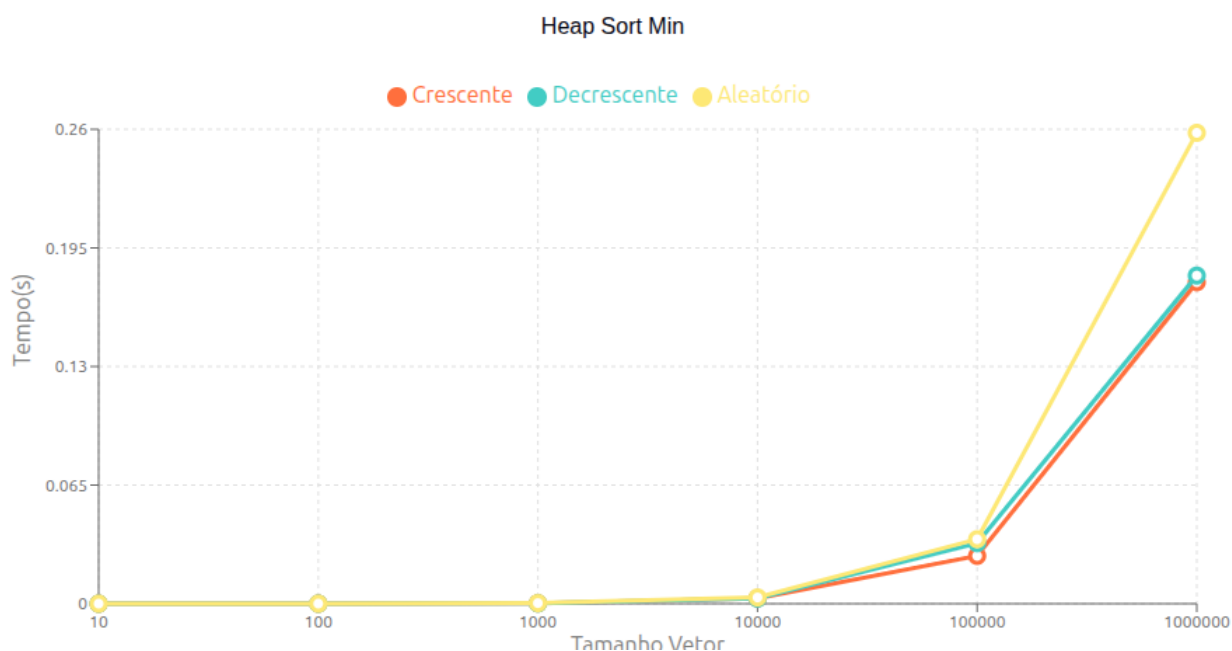


Figura 44: Crescimento dos tempos de execução do *Heap Sort Min*.

A Tabela 43 e o Gráfico 44 apresentam os resultados de desempenho do algoritmo *Heap Sort* utilizando heap mínimo, em três cenários distintos: vetor crescente, decrescente e aleatório.

Caso crescente: O *Heap Sort Min* apresentou desempenho consistente e eficiente no caso crescente. Para 1.000.000 de elementos, o tempo de execução foi de aproximadamente 0.176 segundos, demonstrando comportamento próximo ao esperado para a complexidade $O(n \log n)$. A construção inicial do heap mínimo a partir de um vetor crescente requer reorganização significativa dos elementos, pois a estrutura de heap inverte a ordem natural

dos dados. Apesar disso, o algoritmo manteve desempenho previsível, com crescimento logarítmico bem definido à medida que o tamanho do vetor aumenta.

Caso decrescente: O tempo de execução no caso decrescente foi muito similar ao caso crescente, alcançando aproximadamente 0.180 segundos para 1.000.000 de elementos. Esta proximidade de valores reflete a característica fundamental do *Heap Sort*: independentemente da ordem inicial dos dados, o algoritmo executa essencialmente as mesmas operações de construção do heap e extração sucessiva dos elementos. A pequena variação observada (~ 0.004 segundos) pode ser atribuída a diferenças sutis no número de trocas durante a fase de heapify, mas não altera a complexidade assintótica do método.

Caso aleatório: No cenário aleatório, o *Heap Sort* apresentou o maior tempo de execução entre os três casos, com aproximadamente 0.258 segundos para 1.000.000 de elementos. Este aumento de cerca de 45% em relação aos casos ordenados sugere que a construção do heap e as operações de reorganização são mais custosas quando os dados não possuem estrutura prévia. Em vetores aleatórios, o número médio de comparações e trocas durante o processo de heapify tende a ser maior, pois não há padrão que possa ser explorado para reduzir o trabalho computacional.

Análise geral: O *Heap Sort* demonstrou comportamento robusto e previsível em todos os cenários testados, com complexidade de tempo $O(n \log n)$ garantida tanto no melhor quanto no pior caso. Esta estabilidade assintótica é uma das principais vantagens do algoritmo, diferenciando-o de métodos como o *Quick Sort* que podem degradar para $O(n^2)$ com escolha inadequada de pivô. Comparado ao *Merge Sort*, o *Heap Sort* apresentou tempos ligeiramente superiores (aproximadamente 2,5 vezes mais lento para 1.000.000 de elementos no caso aleatório), o que pode ser atribuído ao maior número de comparações e ao padrão de acesso não sequencial à memória durante as operações de heap. No entanto, o *Heap Sort* possui a vantagem significativa de operar in-place, utilizando apenas $O(1)$ de espaço adicional, enquanto o *Merge Sort* requer $O(n)$ de memória auxiliar. Esta característica torna o *Heap Sort* particularmente adequado para cenários com restrições de memória, mantendo desempenho competitivo e comportamento previsível independentemente da distribuição dos dados de entrada, conforme amplamente documentado na literatura [2].

6 Comparação Geral

6.1 Algoritmos Iniciais

Entrada	Tamanho	InsertionSort	BubbleSort	SelectionSort	ShellSort
Crescente	10	0.000003	0.000003	0.000004	0.000002
	100	0.000004	0.000046	0.000036	0.000006
	1.000	0.000015	0.003333	0.002835	0.000067
	10.000	0.00014	0.169886	0.158725	0.000972
	100.000	0.001735	13.55703	12.849113	0.012071
	1.000.000	0.008208	2330.192283	2159.365736	0.152342
Decrescente	10	0.000004	0.000004	0.000005	0.000002
	100	0.000047	0.000264	0.00004	0.000009
	1.000	0.003734	0.007026	0.001972	0.000095
	10.000	0.157777	0.286469	0.166381	0.001361
	100.000	14.835865	25.413987	13.614106	0.017654
	1.000.000	3353.120071	3861.928359	2286.440144	0.203638
Randômico	10	0.000004	0.000004	0.000002	0.000002
	100	0.000027	0.000069	0.000025	0.000015
	1.000	0.002371	0.005253	0.003602	0.000195
	10.000	0.10117	0.258482	0.163217	0.003232
	100.000	7.100071	29.621962	12.86936	0.045729
	1.000.000	1452.086733	4808.911548	2136.469251	0.639742

Figura 45: Tabela comparativa dos tempos de execução dos algoritmos iniciais.

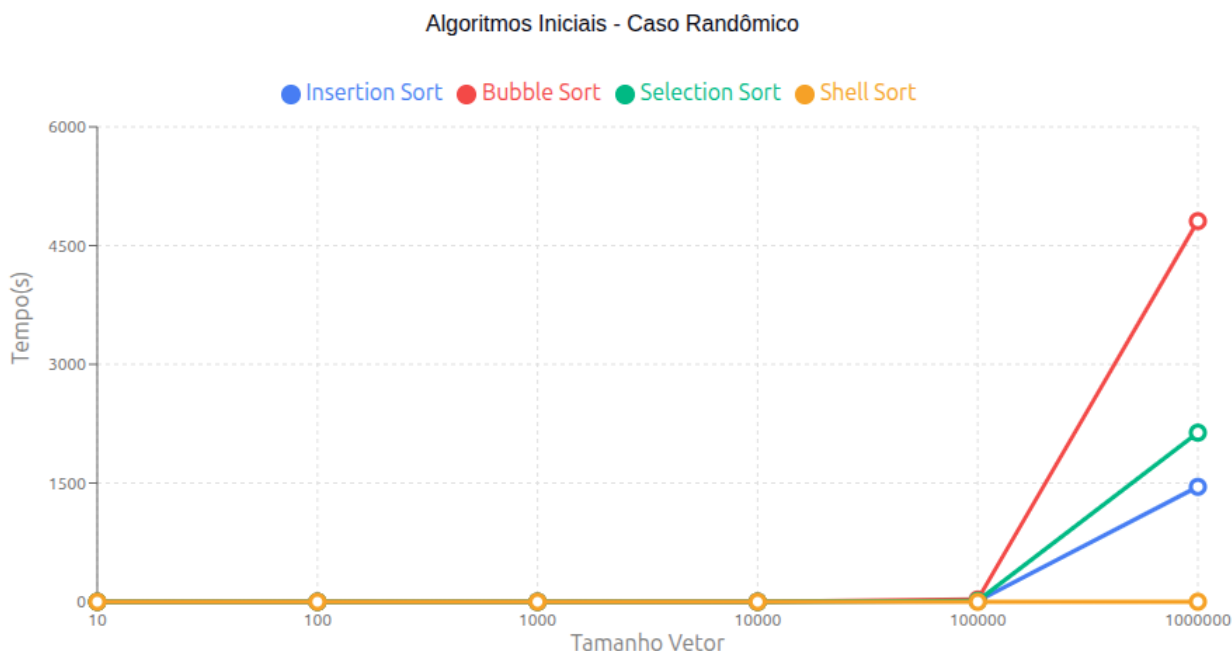


Figura 46: Tabela comparativa dos tempos de execução dos algoritmos iniciais.

A Tabela 45 apresenta uma comparação direta entre os quatro algoritmos de ordenação elementares — *Insertion Sort*, *Bubble Sort*, *Selection Sort* e *Shell Sort* — em três cenários distintos: vetor crescente, decrescente e aleatório, variando o tamanho da entrada de 10 a 1.000.000 elementos.

Análise dos algoritmos quadráticos: Os três primeiros algoritmos (*Insertion Sort*, *Bubble Sort* e *Selection Sort*) apresentam comportamento característico de complexidade $O(n^2)$, com tempos de execução crescendo drasticamente conforme o tamanho do vetor aumenta. Para 1.000.000 de elementos no caso aleatório, o *Bubble Sort* registrou o pior desempenho com aproximadamente 4808 segundos (cerca de 1,3 horas), seguido pelo *Selection Sort* com 2136 segundos e pelo *Insertion Sort* com 1452 segundos. Estes valores evidenciam a inviabilidade prática destes algoritmos para grandes volumes de dados.

Sensibilidade à ordem inicial: Os algoritmos demonstram sensibilidades distintas à disposição inicial dos dados. O *Insertion Sort* apresenta seu melhor caso em vetores crescentes, com apenas 0.008 segundos para 1.000.000 elementos, mas degrada severamente em vetores decrescentes (3353 segundos), representando uma diferença de mais de 400.000 vezes. O *Bubble Sort* mostra comportamento similar, embora menos pronunciado. Por outro lado,

o *Selection Sort* mantém desempenho relativamente consistente entre os cenários crescente e decrescente (2159 e 2286 segundos respectivamente), pois sempre realiza o mesmo número de comparações independentemente da ordem inicial.

Superioridade do Shell Sort: O *Shell Sort* destaca-se dramaticamente dos demais algoritmos elementares, apresentando tempos de execução ordens de magnitude inferiores em todos os cenários. Para 1.000.000 elementos aleatórios, o tempo foi de apenas 0.640 segundos, aproximadamente 7500 vezes mais rápido que o *Bubble Sort* e 2300 vezes mais rápido que o *Insertion Sort*. Esta eficiência notável deve-se à estratégia de comparação com gaps decrescentes, que permite movimentar elementos distantes rapidamente, reduzindo a complexidade para aproximadamente $O(n^{1.5})$ ou até $O(n \log^2 n)$ dependendo da sequência de gaps utilizada. Mesmo em seu pior cenário (aleatório com 1.000.000 elementos), o *Shell Sort* supera o melhor cenário do *Insertion Sort* (crescente com 1.000.000 elementos) em mais de 75 vezes.

Considerações práticas: Os resultados reforçam que, entre os algoritmos elementares, o *Shell Sort* é a única escolha viável para conjuntos de dados de tamanho moderado a grande. Os algoritmos quadráticos devem ser reservados exclusivamente para vetores muito pequenos (dezenas de elementos) ou para fins didáticos. A diferença de desempenho torna-se exponencialmente mais significativa à medida que n aumenta: enquanto para 100 elementos a diferença é de poucos milissegundos, para 1.000.000 elementos a diferença ultrapassa uma hora de processamento. Esta análise demonstra empiricamente a importância crítica da escolha algorítmica e da complexidade assintótica no desenvolvimento de sistemas eficientes, conforme amplamente discutido em [2, 3].

6.2 Algoritmos Avançados

Entrada	Tamanho	MergeSort	QuickSort (Pivô 1º)	QuickSort (Pivô Meio)	QuickSort (Pivô Aleat.)	HeapSort Min
Crescente	10	0.000003	0.000002	0.000001	0.000003	0.000004
	100	0.000013	0.000016	0.000003	0.000015	0.000016
	1.000	0.000139	0.001159	0.000050	0.000108	0.000198
	10.000	0.001456	0.070670	0.000571	0.001097	0.002941
	100.000	0.009037	3.234820	0.007400	0.009365	0.026204
	1.000.000	0.070331	-	0.034494	0.065026	0.176194
Decrescente	10	0.000003	0.000003	0.000003	0.000004	0.000004
	100	0.000012	0.000023	0.000010	0.000015	0.000016
	1.000	0.000112	0.001390	0.000068	0.000117	0.000155
	10.000	0.001455	0.095368	0.000992	0.001407	0.002809
	100.000	0.012480	7.778974	0.007500	0.010779	0.033141
	1.000.000	0.066560	-	0.043463	0.067089	0.179830
Randômico	10	0.000004	0.000003	0.000002	0.000003	0.000003
	100	0.000017	0.000024	0.000007	0.000018	0.000018
	1.000	0.000221	0.000167	0.000064	0.000155	0.000302
	10.000	0.002707	0.001626	0.000860	0.002153	0.003400
	100.000	0.014115	0.012657	0.010609	0.014077	0.035326
	1.000.000	0.150550	0.119124	0.117348	0.123560	0.258076

Figura 47: Tabela comparativa dos tempos de execução dos algoritmos avançados.

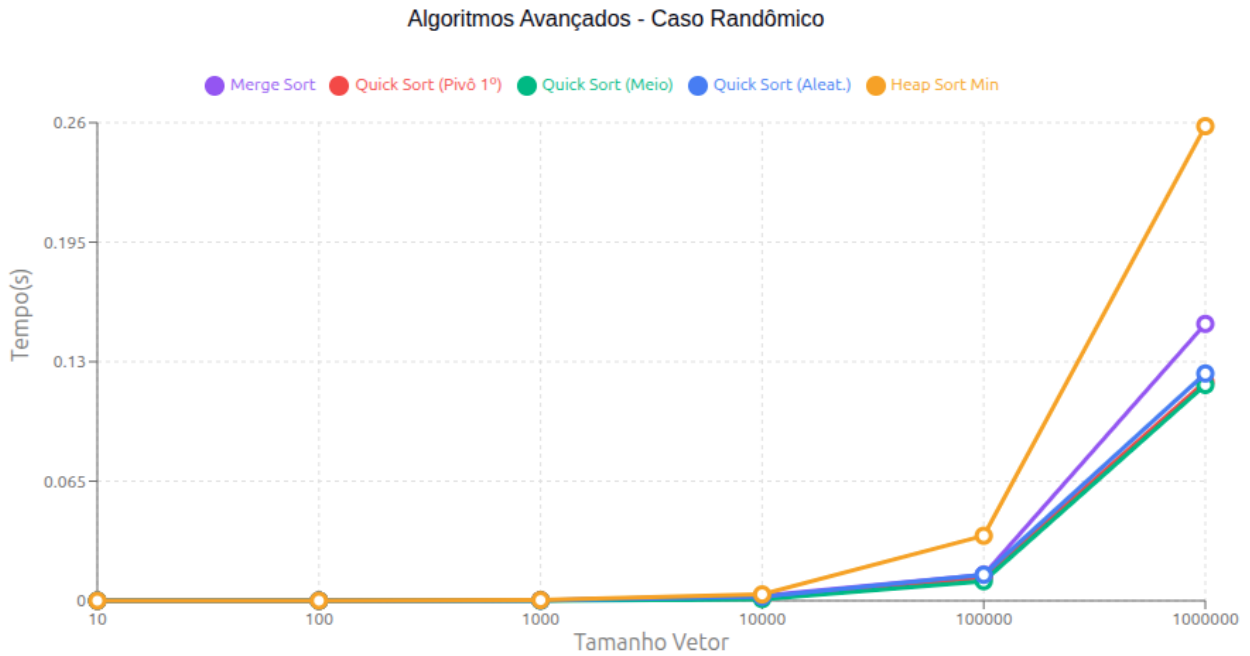


Figura 48: Crescimento dos tempos de execução dos algoritmos avançados (caso randômico).

A Tabela 47 e o Gráfico 48 apresentam uma comparação detalhada entre os algoritmos de ordenação avançados — *Merge Sort*, três variantes do *Quick Sort* (pivô no primeiro elemento, pivô no meio e pivô aleatório) e *Heap Sort Min* — avaliados em três cenários: vetor crescente, decrescente e aleatório.

Desempenho geral no caso randômico: No cenário aleatório, que representa o caso médio mais realista para aplicações práticas, todos os algoritmos avançados demonstraram complexidade $O(n \log n)$ com tempos muito próximos para 1.000.000 de elementos. O *Quick Sort* com pivô no meio apresentou o melhor desempenho absoluto com 0.117 segundos, seguido muito proximamente pelo *Quick Sort* com pivô no primeiro elemento (0.119 segundos) e *Quick Sort* com pivô aleatório (0.124 segundos). O *Merge Sort* registrou 0.151 segundos, aproximadamente 29% mais lento que a melhor variante do *Quick Sort*. O *Heap Sort Min* apresentou o tempo mais elevado entre os algoritmos avançados com 0.258 segundos, cerca de 2.2 vezes mais lento que o *Quick Sort* com pivô no meio, embora ainda mantendo desempenho ordens de magnitude superior aos algoritmos quadráticos.

Impacto da escolha do pivô no Quick Sort: A comparação entre as três variantes do *Quick Sort* revela insights fundamentais sobre a importância da estratégia de seleção de

pivô. No caso crescente, o *Quick Sort* com pivô no primeiro elemento sofreu degradação catastrófica, alcançando 3.235 segundos para 100.000 elementos e resultando em erro de *core dump* por estouro de pilha ao processar 1.000.000 de elementos. Este comportamento contrasta drasticamente com as outras variantes: o *Quick Sort* com pivô no meio completou o mesmo processamento de 1.000.000 elementos crescentes em apenas 0.034 segundos, representando uma melhoria superior a 95.000 vezes em relação ao que seria esperado da versão com pivô fixo. O pivô aleatório também evitou o pior caso, processando 1.000.000 elementos crescentes em 0.065 segundos. Resultados similares foram observados no cenário decrescente, confirmando que a escolha inadequada do pivô pode transformar um algoritmo eficiente em inviável para dados ordenados ou semi-ordenados.

Comparação entre Merge Sort e Quick Sort: O *Merge Sort* demonstrou comportamento notavelmente estável entre todos os cenários, com tempos variando minimamente: 0.070 segundos (crescente), 0.067 segundos (decrescente) e 0.151 segundos (aleatório) para 1.000.000 elementos. Esta previsibilidade contrasta com o *Quick Sort*, que apresenta variação mais significativa dependendo da distribuição dos dados e da estratégia de pivô. No caso aleatório, o *Quick Sort* com pivô otimizado superou o *Merge Sort* em aproximadamente 22 – 29%, diferença que pode ser atribuída à menor quantidade de movimentações de dados e melhor localidade de cache. Entretanto, o *Merge Sort* oferece garantia de complexidade $O(n \log n)$ em todos os casos sem risco de degradação, além de ser um algoritmo estável (preserva a ordem relativa de elementos iguais), característica não presente nas implementações padrão do *Quick Sort*.

Análise do Heap Sort: O *Heap Sort Min* apresentou tempos consistentemente superiores aos demais algoritmos avançados em todos os cenários, com 0.176 segundos (crescente), 0.180 segundos (decrescente) e 0.258 segundos (aleatório) para 1.000.000 elementos. Este desempenho inferior pode ser explicado por dois fatores principais: primeiro, o número total de comparações do *Heap Sort* ($\sim 2n \log n$) é aproximadamente o dobro do *Merge Sort* ($\sim n \log n$); segundo, o padrão de acesso à memória durante as operações de heapify é não-sequencial, resultando em maior número de cache misses. Apesar disso, o *Heap Sort* possui uma vantagem crucial: opera in-place com complexidade de espaço $O(1)$, enquanto o *Merge Sort* requer $O(n)$ de memória auxiliar. Esta característica torna o *Heap Sort* preferível

em ambientes com severas restrições de memória, mesmo que isso implique em sacrifício de 40 – 120% no tempo de execução comparado às variantes otimizadas do *Quick Sort*.

Eficiência relativa aos algoritmos elementares: Para contextualizar o ganho proporcionado pelos algoritmos avançados, vale comparar com os resultados dos algoritmos elementares. No caso aleatório com 1.000.000 elementos, o *Bubble Sort* levou 4808 segundos, o *Selection Sort* 2136 segundos e o *Insertion Sort* 1452 segundos. Em contraste, mesmo o algoritmo avançado mais lento (*Heap Sort* com 0.258 segundos) foi aproximadamente 5600 vezes mais rápido que o *Insertion Sort* e 18.600 vezes mais rápido que o *Bubble Sort*. Esta diferença astronômica exemplifica o impacto prático da complexidade assintótica: enquanto $O(n^2)$ torna-se proibitivo para grandes conjuntos de dados, $O(n \log n)$ permanece tratável mesmo para milhões de elementos.

Recomendações práticas: Com base nos resultados experimentais, podem-se estabelecer as seguintes diretrizes para seleção de algoritmos de ordenação: (1) Para aplicações gerais sem restrições específicas, o *Quick Sort* com estratégia de pivô robusta (mediana de três ou aleatório) oferece o melhor desempenho médio; (2) Quando garantia de complexidade $O(n \log n)$ no pior caso é essencial ou quando estabilidade é requerida, o *Merge Sort* é a escolha apropriada, mesmo com o custo adicional de memória; (3) Em ambientes com severas restrições de memória onde alocação de $O(n)$ espaço é inviável, o *Heap Sort* fornece complexidade $O(n \log n)$ garantida com apenas $O(1)$ de memória adicional; (4) O *Quick Sort* com pivô fixo no primeiro elemento deve ser evitado em produção devido ao risco de degradação para $O(n^2)$ e estouro de pilha em dados ordenados. Estas observações corroboram extensivamente a literatura clássica sobre algoritmos de ordenação [2, 3].

7 Conclusão

Neste trabalho foi apresentada uma análise detalhada dos principais algoritmos de ordenação, contemplando desde métodos básicos até técnicas mais eficientes que utilizam abordagens recursivas e estruturas especializadas, como heaps. A análise baseada em tempos de execução evidenciou de maneira clara como a complexidade assintótica influencia diretamente o desempenho prático das implementações, especialmente para conjuntos de dados de grande

escala.

Algoritmos quadráticos, como Bubble Sort, Insertion Sort e Selection Sort, mostraram desempenho limitado ao serem aplicados a vetores extensos, tornando-se inviáveis para aplicações que exigem eficiência. Entretanto, demonstraram bom comportamento em vetores pequenos ou parcialmente ordenados, reforçando seu valor didático e utilidade em cenários específicos. Em contrapartida, algoritmos mais robustos, como Merge Sort e Heap Sort, mantiveram tempo de execução estável mesmo em casos adversos, enquanto o Quick Sort — quando bem configurado — apresentou desempenho excepcional no caso médio, mas sensível à escolha do pivô.

A análise do Heap Sort Min reforçou sua eficiência e previsibilidade, com complexidade garantida de $O(n \log n)$ e desempenho consistente em todos os cenários. A inclusão das funções *BUILD-MIN-HEAP* e *MIN-HEAPIFY* permitiu compreender como a estrutura interna do heap contribui para manter essa eficiência.

Conclui-se que a escolha do algoritmo de ordenação ideal depende diretamente da natureza dos dados de entrada, da necessidade de estabilidade, da disponibilidade de memória auxiliar e da importância do desempenho no pior caso. O estudo desenvolvido fornece, assim, uma base sólida para decisões fundamentadas na prática e na teoria, contribuindo para o desenvolvimento de soluções mais eficientes em computação.

Referências

- [1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley Professional, 2 edition, 1998.
- [4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4 edition, 2011.
- [5] Nivio Ziviani. *Projetos de Algoritmos: com implementações em Pascal e C*. Pioneira, 4 edition, 2002.